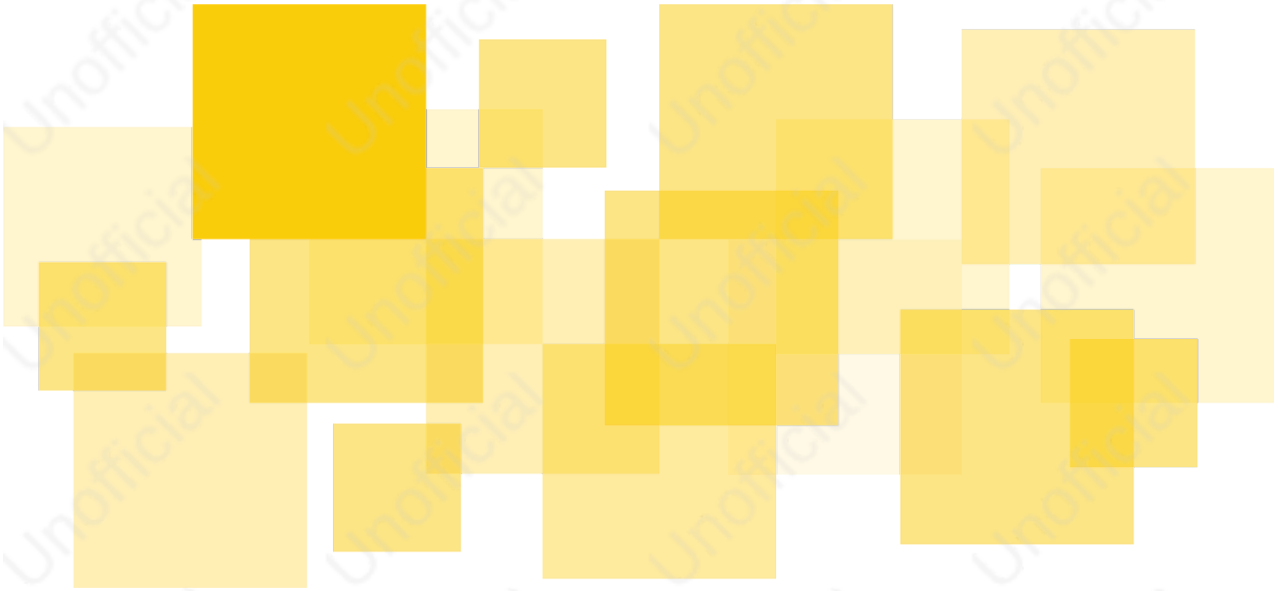
Security Audit Report

Optimism Interop Report Ethereum

Delivered: Not published



Unofficial

Table of Contents

- Disclaimer
- Executive Summary
- Goal
- Methodology
  - Design Review
  - Code Review
  - Fuzzing
- Design Review
  - Supernode Overview
    - Architecture
    - Cross-Validation Logic
      - Supernode State and Invariants
      - Assumptions on the L1 and L2 Chain
      - State Changes from Cross-Validation
      - Implementation Details
  - Interop Filter Overview
    - Architecture
    - Modes of Operation
      - Failsafe Mode
      - Normal Mode
    - Differences to Supernode
- Fuzzing
  - Preliminary Analysis
    - Interop
      - Tested Functions
      - `Interop.progressAndRecord`
      - `Interop.verifyInteropMessages` and `Interop.verifyCycleMessages`
      - Test Doubles
    - Interop Filter
      - Tested Functions
      - Test Doubles

- Fuzzing Implementation
  - Methodology
  - ChainRandomizer improvements
    - `Implementing ChainContainer`
    - `Block time awareness`
    - `Removal of the global cutoff / head-tracking model`
    - `Conversion to member methods`
    - `Simplified cycle generation`
    - `Inline invalidation with probability`
  - Fuzz Target Overview
    - `requireLogsDBChainIntegrity`
    - `requireVerifiedDBChainIntegrity`
  - Chain Randomizer Overview
  - Coverage Report
- Findings
  - Supernode Findings
    - `earliestL1Inclusion` calculated by taking lowest block instead of highest
      - Status
    - `BlockNumberToTimestamp` duplication and inconsistencies
      - Status
    - `SyncStatus()` can silently fail and return empty result
      - Status
    - Supernode missing consistency checks to detect and recover from reorgs
      - Recommendation
      - Status
    - Operations that modify logs DB should leave contents untouched in case of failure
    - Rewind is triggered even if previous verified L1 is still valid
      - Recommendation
    - Unnecessary rewind of the executing engine in `DecisionRewind` case
    - Inaccurate debug message in `verifyCanAddTimestamp()` when `seal.Timestamp == ts`
    - Construction of synthetic block might fail if block execution depends on fee recipient

        - Recommendation
    - Block invalidation may fail silently without rewinding executing engine
    - `l1Inclusion()` might retrieve inconsistent L1 block in the case of reorg
- Interop Filter Findings
    - `LockstepCrossValidator` never advances if `startTimestamp` is `0`
        - Recommendation
        - Status
    - Integer undeflow in `calculateStartingBlock`
        - Recommendation
        - Status
    - Unchecked edge cases in `findAndSetEarliestBlock`
        - Analysis
            - Case 1: `OpenBlock(latestBlock)` fails
            - Case 2: `latestBlock == 0`
        - Call site
        - Recommendation
    - Ingestion continues after setting error state
        - Analysis
        - Recommendation
- Informative Findings
    - `LockstepCrossValidator` informational findings
        - Consider enforcing `inclusionTimestamp <= execTimestamp` in `validateMessageTiming` if `timeout > 0`
        - Consider checking for ingester errors in `getMinIngestedTimestamp`
        - `startTimestamp` is assumed cross-valid at initialization
    - `LogsDBChainIngester` informational findings
        - Consider reordering check and function call in `processBlockLogs`
        - Missing `nil` check for `rollupCfg` in `NewLogsDBChainIngester`
        - `ethClient` and `rpcClient` are not closed if `logsDB.Close()` returns an error
        - `SetError` discards `IsUint64` result on conversion
        - Temporary directory is not cleaned up if `LogsDB` creation fails
        - Consider swapping condition order

        - Consider using a write lock in `findAndSetEarliestBlock`
        - Consider refactoring `sealParentBlock`
        - Consider sharing the `LogsDB` interface between `op-supernode` and `op-interop-filter`
        - Consider extracting a function for the DB resume logic in `initIngestion`
        - Inconsistent computation of starting block in `initIngestion`
        - `hasLatest` should be asserted instead of branched on in `ingestBlock`
        - Consider implementing `findAndSetEarliestBlock` in terms of `LogsDB.FirstSealedBlock`
        - Avoid subtraction in `runIngestion`
        - `checkReorg` logs spurious error if `earliestIngestedBlock` is unset
        - No output if the error is set
        - Consider adding a `Rewind()` method
        - Document in `processBlockLogs` that `execMsg == nil` is a valid case
    - Supernode supports same-timestamp messages, but Interop-Filter rejects them
    - Supernode code quality suggestions
        - `interop.go`
        - `verified_db.go`
        - `algo.go`
        - `rewind.go`
- Appendix
    - `LockstepCrossValidator` Function Summaries
        - Call Graph
        - Function Summaries
            - `validateMessageTiming`
            - `getMinIngestedTimestamp`
            - `CrossValidatedTimestamp`
            - `ValidateAccessEntry`
            - `validateExecutingMessage`
            - `validateTimestamp`
            - `advanceValidation`
    - `LogsDBChainIngester` Function Summaries
        - Call Graph
        - Function Summaries
            - `NewLogsDBChainIngester`
            - `Start`
            - `Stop`

    - `Ready`
    - `SetError`
    - `Error`
    - `ClearError`
    - `Contains`
    - `LatestBlock`
    - `BlockHashAt`
    - `LatestTimestamp`
    - `GetExecMsgsAtTimestamp`
    - `findAndSetEarliestBlock`
    - `calculateStartingBlock`
    - `initLogsDB`
    - `runIngestion`
    - `initIngestion`
    - `checkReorg`
    - `sealParentBlock`
    - `ingestBlock`
    - `processBlockLogs`
- Dafny Model of the Supernode
    - Components
    - Caveats

Disclaimer

This report does not constitute legal or investment advice. You understand and agree that this report relates to new and emerging technologies and that there are significant risks inherent in using such technologies that cannot be completely protected against. While this report has been prepared based on data and information that has been provided by you or is otherwise publicly available, there are likely additional unknown risks that otherwise exist. This report is also not comprehensive in scope, excluding a number of components critical to the correct operation of this system. This report is for informational purposes only and is provided on an "as-is" basis, and you acknowledge and agree that you are making use of this report and the information contained herein at your own risk. The preparers of this report make no representations or warranties of any kind, either express or implied, regarding the information in or the use of this report and shall not be liable to you or any third parties for any acts or omissions undertaken by you or any third parties based on the information contained herein.

Smart contracts are still a nascent software arena, and their deployment and public offering carry substantial risk.

Finally, the possibility of human error in the manual review process is very real, and we recommend seeking multiple independent opinions on any claims that impact a large quantity of funds.

Unofficial

Executive Summary

Optimism engaged Runtime Verification Inc. to conduct a comprehensive security audit of the OP Extension, focusing on the Supernode and Interop Filter components. The audit began on February 9, 2026, and was scheduled to conclude on April 13, 2026, spanning a total of nine weeks. It was subsequently extended by one additional week to account for planned conference absences. The objective of the audit is to evaluate the correctness, reliability, and security of these implementations, identify vulnerabilities that may compromise system integrity, and provide actionable recommendations to strengthen the protocol.

Commit `be00aaa2cc3c8bea80e916d17c30a2b5d8d87696` was used as the initial frozen commit for the audit, but it was later updated to `bd29b7d309249be9e9032f6c2663829edf064e1d` in order to incorporate changes to the design addressing the issues described in finding Supernode missing consistency checks to detect and recover from reorgs.

The audit is structured in three phases: Design Review, Code Review, and a Fuzzing Campaign. Runtime Verification's process combines manual code inspection with formal verification techniques, including invariant derivation, property-based fuzz testing, and systematic analysis of adversarial execution scenarios, to rigorously analyze the behavior of the Supernode and Interop Filter across a wide range of conditions.

The Supernode and Interop Filter are new components of the OP Stack that extend Optimism's interoperability model by managing cross-safe and cross-unsafe verification of messages across L2 chains, respectively. Together, they enable blocks to be classified at different safety levels based not only on their availability on L1, but also on whether their cross-chain dependencies have been validated.

The audit has led to the identification of several findings, which are detailed in the subsequent sections of this report, with proposed mitigations to improve the robustness, security, and maintainability of the Supernode and Interop Filter. Where applicable, responses and fixes from the client team have been reviewed and incorporated into the final assessment.

Goal

The goals of the audit is include:

- Review the high-level business logic and intended behavior of the Supernode and Interop Filter based on the provided documentation, architecture, and system goals;
- Review the low-level implementation details of both codebases, with particular focus on the cross-safe verification logic in the Supernode and the cross-unsafe verification mechanism in the Interop Filter;
- Analyze the correctness and reliability of core mechanisms such as executing message validation, same-timestamp cycle detection, L2 rewind procedures, failsafe mode enforcement, and component boundary assumptions;
- Identify opportunities to improve documentation clarity, expose edge-case bugs, and verify the correctness of internal logic through manual review, invariant analysis, and fuzz testing.

The audit focuses on identifying potential vulnerabilities and logical flaws that could lead to incorrect cross-chain message validation, unsafe block classification, improper failsafe behavior, or violation of safety and liveness properties. Particular attention is given to failure modes that may arise at the boundary between the components, as well as from incomplete data validation, incorrect state cleanup, or improper enforcement of system invariants. Additionally, the audit includes recommendations aimed at improving code maintainability, documentation quality, and overall robustness of both components.

Methodology

We began the security engagement with a 3-week design review where we outlined the structure of the Supernode and Interop Filter, formally defined the critical concepts on which they are built, and modeled their high-level behavior. The formal definitions and model produced during this phase were reviewed with the OP team to ensure their correctness. In the transition between this phase and the next, we outlined the key properties that the implementation must satisfy to be consistent with the design and ensure the safety of the system. These properties were then used to guide both the code review and fuzz testing phases of the engagement.

The Supernode is responsible for managing cross-safe verification across L2 chains in the OP network, coordinating chain state through components such as the ChainContainer, VirtualNode, and EngineController, and tracking the verification results through components such as the VerifiedDB and DenyList. The Interop Filter complements this by providing cross-unsafe verification through the LockstepCrossValidator, enabling earlier classification of cross-chain message validity prior to full cross-safe confirmation. Together, these components extend Optimism's interoperability model by ensuring that executing messages are validated consistently across chains, supporting both safety and liveness properties of the broader protocol.

As agreed with the OP team, our primary focus during this engagement is on the cross-safe and cross-unsafe verification mechanisms, the correctness of executing message validation, and the assumptions and interactions at the interface with the L1 and L2 chains.

Design Review

- Review the existing documentation of the Supernode and Interop Filter to understand essential terms and concepts.
- Document all components of each system and their interactions, including ChainContainer, VirtualNode, EngineController, VerifiedDB, DenyList, LockstepCrossValidator, and LogsDBChainIngester.
- Document and formalize the cross-validation mechanisms in both components, defining the conditions under which executing messages are considered cross-unsafe and cross-safe.
- Outline a model of the Supernode's internal mechanisms used to determine cross-validity, including same-timestamp cycle detection, L2 rewind procedures, and DenyList behavior.
- Outline a model of the Interop Filter's validation logic, including failsafe mode activation and deactivation and the role of the LockstepCrossValidator.
- Document relevant assumptions and invariants of the model, including assumptions about L1 and L2 behavior, component boundary contracts, and expected error handling.
- Identify issues and points of improvement in the documentation.

Property Specification

- Formalize the correctness properties for cross-safe and cross-unsafe verification in both components, ensuring that executing messages are neither accepted unduly nor incorrectly rejected.
- Verify that safety and liveness properties are maintained across all main execution paths, including successful cross-validation, failed cross-validation, L2 sync waiting, and L1 reorg handling.
- Examine whether there are any conditions under which the Supernode or Interop Filter could stop making progress or reach an unrecoverable state.

Code Review

- Analyze the main execution paths of the Supernode and verify that protocol invariants are maintained throughout, with particular attention to intermediate state cleanup on validation errors, the L2 rewind procedure, and the DenyList implementation.
- Review the Interop Filter implementation for consistency with the cross-validation logic of the Supernode, correctness of failsafe mode behavior, and the behavior of the LogsDBChainIngester when reading from the L2 chain and managing the logs database.
- Carefully analyze the interface between components, identifying incorrect or unsafe assumptions across component boundaries.
- Report any implementation-level bugs or code vulnerabilities found, including low-level concerns such as arithmetic overflow, out-of-bounds access, and concurrency issues.
- Identify fuzzing targets informed by the findings and high-risk interactions observed during the review.

Fuzzing

- Write fuzzing targets using Go to test properties and look for crashes, invariant violations, and other unexpected behaviors in both the Supernode and the Interop Filter.
- Build a reusable harness for initializing each component with randomized valid state as a starting point for the tests, building on fuzzing infrastructure from previous engagements where applicable.
- Implement fuzz tests for the core interop verification logic, including Interop.progressAndRecord, Interop.verifyInteropMessages, and Interop.verifyCycleMessages in the Supernode, and LockstepCrossValidator.advanceValidation and LockstepCrossValidator.ValidateAccessEntry in the Interop Filter.

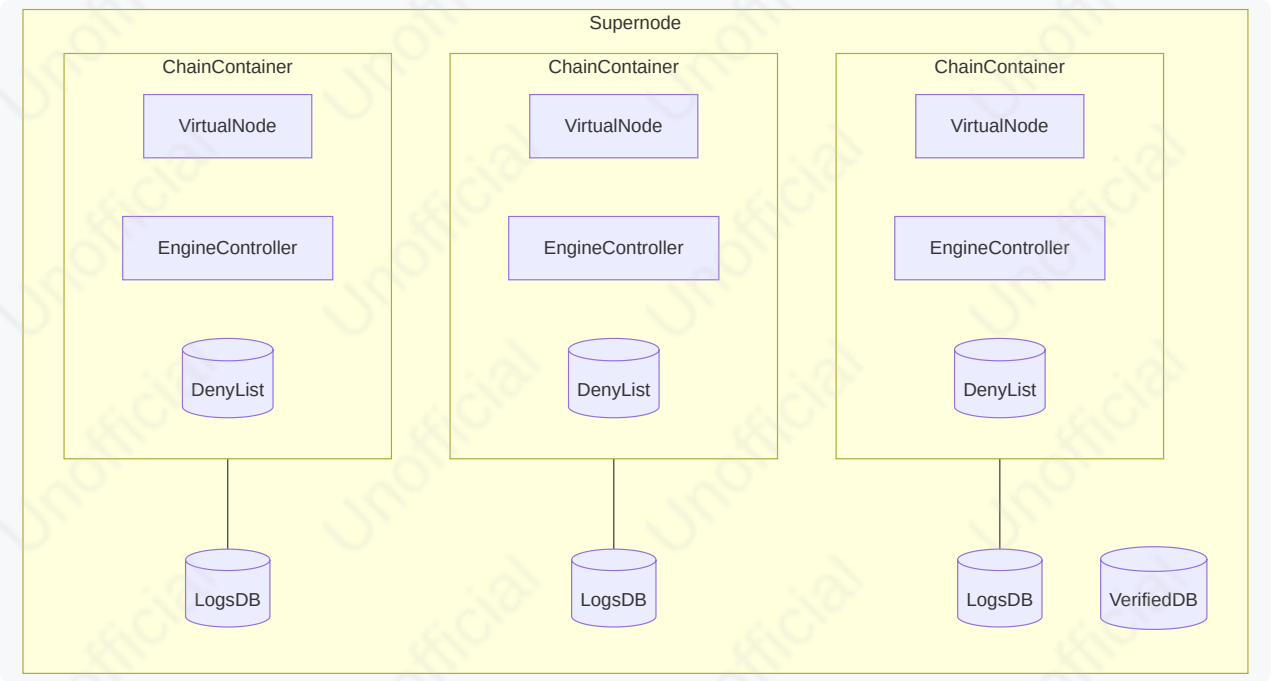
Design Review

This section presents the overview of the design review conducted on the Supernode and Interop Filter systems.

Unofficial

Supernode Overview

Architecture



A `Supernode` includes the following components:

- A `VerifiedDB` storing the cross-validity history. The `VerifiedDB` contains an entry for every timestamp for which cross-validation has been performed. This entry is a `VerifiedResult` consisting of:
  - The verified timestamp t.
  - The L2 heads for each L2 chain at that timestamp. Since an L2 might not have a block at every timestamp, this is the highest block with timestamp $\leq t$.
  - The L1 head at which cross-validation at that timestamp can be performed. Note that each L2 head might be derived from a different L1 block, so the L1 head is the highest among those. In other words, it's the earliest L1 block at which all L2 heads are available.
- A `ChainContainer` for each L2 chain, itself including the following components:
  - An `EngineController`, which serves as an interface to the L2's execution layer. When cross-validation fails for a given timestamp, the `EngineController` for all affected chains rewinds the

execution layer to the preceding timestamp by performing a fork choice update. This allows the execution layer to continue building the L2 chain on top of the most recent valid block.
    - A `VirtualNode`, an in-memory `OpNode` instance used to derive the Local Safe chain from L1 batch data. When checking cross-validity for a given timestamp, the `Supernode` queries each L2's `VirtualNode` to retrieve the logs up to the target timestamp. `VirtualNode`s are intented to be fungible: when cross-validation detects an invalid Executing Message, invalidating the Local Safe chain, the `VirtualNode` is destroyed and recreated after the `EngineController` has been reset to the previous timestamp and the invalid blocks have been added to the `DenyList`. This forces the `VirtualNode` to rederive the Local Safe chain, this time avoiding the newly-invalidated blocks.
    - A `DenyList` recording the blocks that have been invalidated during cross-validation. This `DenyList` is consulted by the `VirtualNode` when deriving the Local Safe chain to ensure that a block that has already been deemed invalid is not revisited. If the `VirtualNode` would derive a block that is present in the `DenyList`, it replaces it with a deposit-only block instead.
- A `LogsDB` for each L2 chain, storing the logs retrieved from the `VirtualNode` up to the latest verified timestamp.

Cross-Validation Logic

The `Supernode` performs cross-validation one timestamp at a time, consistently incrementing the timestamp by 1 each time. If all L2 chains are caught up to the target timestamp, it downloads the logs for this timestamp and checks the Executing Messages for cross-validity. If they pass validation, the blocks are recorded in the `VerifiedDB`. Otherwise, the invalid blocks are recorded in the `DenyList` for their respective chains and the invalidated chains are rewound to the previous timestamp.

Supernode State and Invariants

After performing cross-validation for timestamp t, and before performing cross-validation for $t+1$, the internal state of the `Supernode` for a set of k L2 chains consists of the following:

- A `LogsDB` $L_j = (B_0^j, \ell_0^j), \ldots, (B_{n_j}^j, \ell_{n_j}^j)$ for each L2 chain j, where B_i^j is a block from chain j and ℓ_i^j is the list of logs of B_i^j.
- A `VerifiedDB` containing the cross-validity history, given by $(t_0, C_{t_0}, C_{t_0}^1, \ldots, C_{t_0}^k), \ldots, (t, C_t, C_t^1, \ldots, C_t^k)$, where t_0 is the activation timestamp, t is the last verified timestamp, and for each timestamp $t_0 \leq i \leq t$:
    - $C_i^1, \ldots, C_i^k$ are the verified L2 heads for timestamp i for each of the k L2 chains.
    - C_i is the verified L1 head for timestamp i.

- A `DenyList` D_j for each L2 chain j, containing blocks from chain j that were invalidated at timestamps $\leq t + 1$.

The following invariants are expected to be true at this state:

- ℓ_i^j is the list of logs for block B_i^j, for all i and j.
- B_i^j is the parent block of B_{i+1}^j, for all i and j.
- Every B_i^j is cross-valid given the cross-validity history. In other words, all Executing Messages in ℓ_i^j are valid:
  - The corresponding Initiating Message exists in the `LogsDB` of its own L2 chain.
  - The Initiating Message's timestamp t' is less than or equal to the timestamp of B_i^j.
  - The Initiating Message hasn't expired (the timestamp of B_i^j is less than or equal $t' + t_{exp}$, where t_{exp} is the expiration time of the Initiating Message).
  - No cycles are formed by messages from the same timestamp.
- $C_{t_0}^j = B_0^j$, for all j.
- $C_t^j = B_{n_j}^j$, for all j.
- C_i^j is either equal to C_{i+1}^j or its parent, for all i and j (consequently, $C_{t_0}^j, \ldots, C_t^j$ is equal to $B_0^j, \ldots, B_{n_j}^j$ with possible repetitions in the middle).
- C_i^j is the highest block on its chain with timestamp $\leq i$. In other words, all children of C_i^j have timestamp $> i$.
- $C_{t_0}, \ldots, C_t$ are all part of the same linear chain (but there might be missing blocks in the middle).
- C_i is the earliest L1 block on its linear chain where all L2 blocks $C_i^1, \ldots, C_i^k$ are available. In other words, C_i is the highest among the L1 blocks from which $C_i^1, \ldots, C_i^k$ were derived.
- $B_i^j \notin D_j$ for any j and i. In other words, a block in the `LogsDB` (and, consequently, in the `VerifiedDB`) does not appear in the `DenyList`.
- For every block $B \in D_j$, the timestamp of B is $\leq t + 1$ (note that the `DenyList` *can* contain invalidated blocks for the next timestamp $t + 1$, added from previous unsuccessful attempts to cross-validate $t + 1$).

At the initial state of the `Supernode`, the `LogsDB`s, `DenyList`s and `VerifiedDB` are all empty. As a consequence, all of the above invariants hold by default in this state.

Assumptions on the L1 and L2 Chain

During a round of cross-validation, the Supernode queries the state of the L1 chain (via an `L1Client`) and L2 chains (via the chain's respective `VirtualNode`). When making these queries, we make the following assumptions about the state and behavior of the L1 and L2 chains:

- Timestamps don't repeat, i.e., there is at most one block per chain per timestamp.
- Each L2 chain j has a deterministic block time t_j, and the timestamp of every block in that chain is exactly t_j higher than the previous block.
- Outside of cross-validation, an L2 chain will only reorg if the L1 chain reorgs. This also means that by the time we are able to observe the results of a reorg in L2, we must necessarily be able to observe it in L1 as well. Note, however, that the reverse is not true: we might be able to observe that the L1 has had a reorg before an L2 chain has been able to sync up.
- Assuming sufficient time between L1 reorgs, each L2 chain will *eventually* sync up to the L1.
- If an L2 chain j tries to derive a block that's in its `DenyList` D_j, it will instead insert a deposit-only block in its place. Since the `VirtualNode` for an L2 chain is always destroyed and recreated after a block is added to the `DenyList`, forcing it to rederive the safe chain, we can guarantee that at the start of a round of cross-validation the safe chain for an L2 will never have a block that's currently in its `DenyList`.
- Technically speaking, L1 and L2 chains can reorg at any point between two rounds of cross-validation, or even in the middle of a round. Because of this, any chain can at any point be inconsistent with the other chains, with the internal state of the Supernode, or even with previous queries made to the same chain. Therefore, we should assume that any queries to the state of the L1/L2s during a round can return an arbitrary result, except for what the assumptions above guarantee.

State Changes from Cross-Validation

The following state changes happen after a round of cross-validation (with last verified timestamp t and next timestamp $t+1$):

1. For each L2 chain j, let B_j be the highest block on safe chain j with timestamp $\leq t+1$. If there should be a higher block with timestamp $\leq t+1$ (note that this can be predicted based on the L2 chain's block time t_j, as per the assumption mentioned above) but it hasn't been derived yet, no state update happens (need to wait for chain j to catch up to the next timestamp $t+1$).
2. Otherwise, let B'_j be the L1 block from from which B_j was derived. If the B'_j are not all in the same linear chain, this means that a reorg has happened and some of the L2s haven't synced. In that case, no state update happens (we wait until next round to give the L2s time to sync up).
3. Otherwise, if C_t is not in the same linear chain as the B'_j, this means that C_t has been reorged out and is no longer a valid L1 block. Therefore, we need to roll back to the previous timestamp to cross-validate again:
   - The cross-validity history in the `VerifiedDB` is pruned to $(t_0, C_{t_0}, C^1_{t_0}, \ldots, C^k_{t_0}), \ldots, (t-1, C_{t-1}, C^1_{t-1}, \ldots, C^k_{t-1})$ by removing the last entry $(t, C_t, C^1_t, \ldots, C^k_t)$.
   - Every `DenyList` D_j is pruned by removing all entries with timestamp $\geq t$ (i.e. t or $t+1$).

- For every chain j such that $C^j_{t-1} \neq B^j_{n_j}$, the `LogsDB` L_j is pruned to $(B^j_0, \ell^j_0), \ldots, (B^j_{n_j-1}, \ell^j_{n_j-1})$ by removing the last entry $(B^j_{n_j}, \ell^j_{n_j})$.

4. Otherwise, since every B_j is consistent with the latest verified L1 block C_t, it must be on the same linear chain as $C^j_t = B^j_{n_j}$. Then, since there is at most one block per timestamp and $B^j_{n_j}$ has timestamp t, either $B_j = B^j_{n_j}$ or $B^j_{n_j}$ is B_j's parent. B_j is invalid if it has an invalid Executing Message or an Executing Message that is part of a cycle. If any B_j is invalid, then
   - Each invalid B_j is added to its respective `DenyList` D_j.
   - For each chain j such that B_j is invalid, the `ChainContainer` is reset by (a) rewinding the `EngineController` to the highest block with timestamp $\leq t$, and (b) destroying and recreating the `VirtualNode`. Since this will force the `VirtualNode` to rederive the safe chain, and now B^j is in the `DenyList`, it will replace the invalid block with a deposit-only block.
   - Note that no `LogsDB` should be updated in this case, so any logs that have been added in the cross-validation process must be removed again.

5. Otherwise (if all B_j are valid), then
   - The cross-validity history is extended to $(t_0, C_{t_0}, C^1_{t_0}, \ldots, C^k_{t_0}), \ldots, (t, C_t, C^1_t, \ldots, C^k_t), (t+1, C_{t+1}, C^1_{t+1}, \ldots, C^k_{t+1})$, where $C^j_{t+1} = B_j$ and C_{t+1} is the highest L1 block among $B'_1, \ldots, B'_k$ (the blocks from which $C^1_{t+1}, \ldots, C^k_{t+1}$ were derived).
   - Every logs database L_j such that $B_j \neq B^j_{n_j}$ is extended to $(B^j_0, \ell^j_0), \ldots, (B^j_{n_j}, \ell^j_{n_j}), (B^j_{n_j+1}, \ell^j_{n_j+1})$, where $B^j_{n_j+1} = B_j$ and $\ell^j_{n_j+1}$ is the list of logs for $B^j_{n_j+1}$.

Note that if we are in the initial state and the databases are empty, instead of $t+1$ we perform cross-validation for the activation timestamp t_0. In this case we can skip the consistency check with C_t on step 3, since there is no verified L1 head C_t.

Implementation Details

The following are implementation details of the Supernode that slightly diverge from this abstracted model:

- When updating the Supernode state in steps 3-5, it's possible that an error might cause a partial update. For example, a transient error might prevent the `LogsDB` from being pruned in step 3 after the `VerifiedDB` has been pruned. In such a case, the state update is stored in the `VerifiedDB` as a `PendingTransition`, and instead of performing the next round of cross-validation the Supernode will re-try applying this `PendingTransition` until it succeeds. This means that, while there is a `PendingTransition`, the Supernode might be in an invalid, partial state, where not all of the invariants listed above hold. As long as all errors that might cause this state are transient, the state

update should eventually succeed, and as long as the state updates of each individual component (`VerifiedDB` , `ChainContainer` s and `LogsDB` s) are implemented in an idempotent way (that is, trying to re-apply a state update that was already applied doesn't change the state), succeeding in re-applying the `PendingTransition` should return the Supernode to a valid state where the invariants hold.

- When the first block B_0 is added to a `LogsDB` and this block is not the genesis block (that is, it has a parent block), a "virtual parent block" is added to the `LogsDB` first. This virtual block has the same hash and number of B_0's actual parent, but no logs or parent hash itself (as if it were a genesis block). It also has the same timestamp as B_0. The reason this "virtual parent block" is necessary is because trying to add logs to the `LogsDB` will throw an error if the parent hash doesn't match the previous block in the database.

Interop Filter Overview

The Interop Filter is a service, independent from the Supernode, for cross-unsafe validation of blocks and transactions.

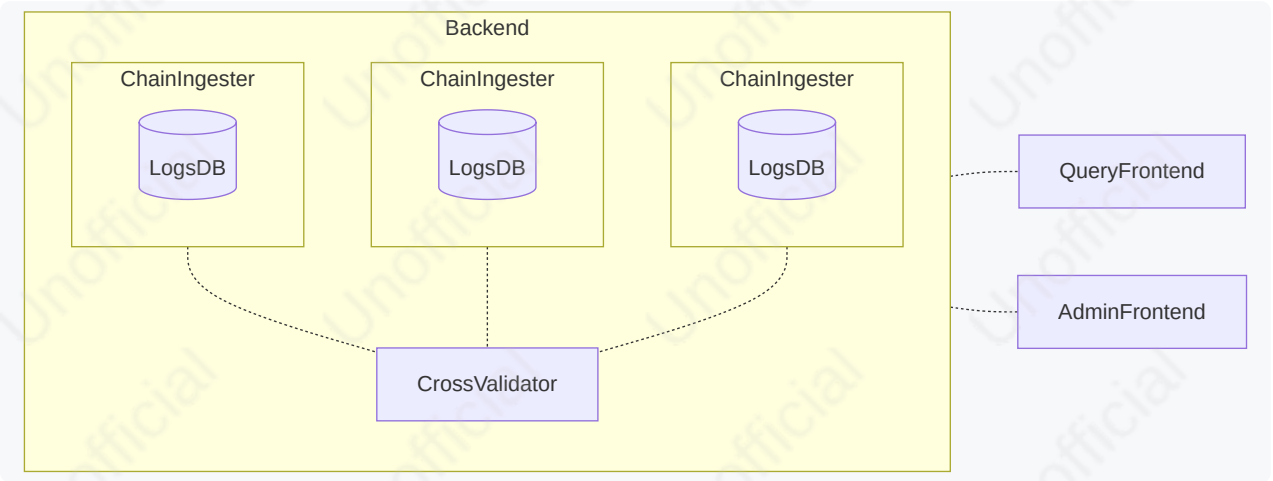
In the following, we give a short description of its architecture, describe its basic modes of operation and highlight differences from Supernode. Our investigation is based on commit https://github.com/ethereum-optimism/optimism/tree/be00aaa2cc3c8bea80e916d17c30a2b5d8d87696.

Our findings are presented in Interop Filter Findings.

Informational findings are described in `LockstepCrossValidator` informational findings and `LogsDBChainIngester` informational findings.

Descriptions for the functions of the `LockstepCrossValidator` and `LogsDBChainIngester` modules, produced during the audit, can be found in the appendices in `LockstepCrossValidator` Function Summaries and `LogsDBChainIngester` Function Summaries, respectively.

Architecture



The Interop Filter includes the following components:

- A `ChainIngester` for each L2 chain, responsible for continuously reading blocks from the respective chain and storing them in a `LogsDB`. This runs asynchronously from the cross-validation loop, and tries to process as many blocks as are available. The `ChainIngester` is also responsible for detecting reorgs in its chain and raising the appropriate error.
- A `CrossValidator`, responsible for cross-validating the data stored across the `LogsDB`s of each chain. Cross-validation proceeds one timestamp at a time, similarly to the Supernode, and tracks the

latest cross-validated timestamp. It can also be queried to validate an executing message present in a transaction's access list, checking that this message is recorded in the corresponding chain's `LogsDB` and that it has been cross-validated.

- Two frontends: a public `QueryFrontend` used for querying the `CrossValidator` to confirm the cross-validity of an access-list entry, and an authenticated `AdminFrontend` exposing methods to enable and disable the failsafe mode. When failsafe mode is enabled, cross-validity queries to the Interop Filter are blocked.

Modes of Operation

Failsafe Mode

In failsafe mode, the `Backend` rejects all `CheckAccessList` queries.

ethereum-optimism/optimism/op-interop-filter/filter/backend.go

Line 132 to 139 in be00aaa

```
132  // CheckAccessList validates the given access list entries.
133  func (b *Backend) CheckAccessList(ctx context.Context, inboxEntries []common.Hash,
134   minSafety types.SafetyLevel, execDescriptor types.ExecutingDescriptor) error {
135
136   if b.FailsafeEnabled() {
137       b.metrics.RecordCheckAccessList(false)
138       return types.ErrFailsafeEnabled
139   }
```

Failsafe mode is active if it has been manually enabled, or if any major component (the `CrossValidator` or any `ChainIngester`) is in the error state.

ethereum-optimism/optimism/op-interop-filter/filter/backend.go

Line 93 to 97 in be00aaa

```
93  // FailsafeEnabled returns true if failsafe is manually enabled OR any chain has an error
94  // OR the cross-validator has an error.
95  func (b *Backend) FailsafeEnabled() bool {
96   return b.manualFailsafe.Load() || len(b.GetChainErrors()) > 0 || b.crossValidator.Error() != nil
97  }
```

It can be manually triggered via `AdminFrontend` .

ethereum-optimism/optimism/op-interop-filter/filter/frontend.go

Line 43 to 47 in be00aaa

```go
// SetFailsafeEnabled enables or disables failsafe mode
func (a *AdminFrontend) SetFailsafeEnabled(ctx context.Context, enabled bool) error {
    a.backend.SetFailsafeEnabled(enabled)
    return nil
}
```

Error State Handling

`CrossValidator`

`CrossValidator` sets its error state in its validation loop if it fails to validate a given timestamp.

ethereum-optimism/optimism/op-interop-filter/filter/lockstep_cross_validator.go

Line 308 to 313 in be00aaa

```go
        // Validate all messages at this timestamp across all chains
        if err := v.validateTimestamp(nextTs); err != nil {
            v.log.Error("Cross-validation failed", "timestamp", nextTs, "err", err)
            v.setError(err.Error())
            return
        }
```

Once in the error state, the cross-validation loop stalls and waits.

ethereum-optimism/optimism/op-interop-filter/filter/lockstep_cross_validator.go

Line 252 to 266 in be00aaa

```go
func (v *LockstepCrossValidator) runValidationLoop() {
    defer v.wg.Done()

    ticker := time.NewTicker(v.validationInterval)
    defer ticker.Stop()

    for {
        select {
        case <-v.ctx.Done():
            return
        case <-ticker.C:
            v.advanceValidation()
        }
    }
}
```

ethereum-optimism/optimism/op-interop-filter/filter/lockstep_cross_validator.go

Line 268 to 273 in be00aaa

```
268 // advanceValidation tries to advance the cross-validated timestamp one step at a time.
269 func (v *LockstepCrossValidator) advanceValidation() {
270  // Stop if we've already hit a validation error
271  if v.Error() != nil {
272      return
273  }
```

For a detailed and more formal description of `CrossValidator` error handling, see `LockstepCrossValidator` Function Summaries.

`ChainIngester`

A `ChainIngester` sets its error state in its ingestion loop if it detects a reorg,

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 551 to 552 in be00aaa

```
551  c.SetError(ErrorReorg, fmt.Sprintf("reorg at height %d: db has %s, chain has %s",
552      headNum, dbHash, head.Hash()))
```

if it fails to ingest a block due to parent hash mismatch,

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 609 to 615 in be00aaa

```
609      // Parent hash of new block must match our latest sealed block
610      if blockInfo.ParentHash() != latestBlock.Hash {
611          c.log.Warn("Detected reorg: parent hash mismatch",
612              "block", blockNum,
613              "expected_parent", latestBlock.Hash,
614              "actual_parent", blockInfo.ParentHash())
615          c.SetError(ErrorReorg, fmt.Sprintf("parent hash mismatch at block %d", blockNum))
```

or if it fails to add a new block and its logs to the `LogsDB`

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 620 to 635 in be00aaa

```
620    logCount, err := c.processBlockLogs(blockInfo, blockID, receipts, blockNum)
621    if err != nil {
622        if errors.Is(err, types.ErrConflict) {
623            c.SetError(ErrorConflict, fmt.Sprintf("database conflict at block %d", blockNum))
624            return nil
625        }
626        if errors.Is(err, types.ErrDataCorruption) {
627            c.SetError(ErrorDataCorruption, fmt.Sprintf("data corruption at block %d: %v", blockNum,
err))
628            return nil
629        }
630        if errors.Is(err, ErrInvalidLog) {
631            c.SetError(ErrorInvalidExecutingMessage, fmt.Sprintf("invalid log at block %d: %v",
blockNum, err))
632            return nil
633        }
634        return err
635    }
```

In all cases, block ingestion stalls while the error state is set.

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 410 to 413 in be00aaa

```
410        // Skip if in error state
411        if c.Error() != nil {
412            continue
413        }
```

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 583 to 585 in be00aaa

```
583    if c.Error() != nil {
584        return nil
585    }
```

An error in any ingester also causes the `CrossValidator` 's validation loop to stall.

ethereum-optimism/optimism/op-interop-filter/filter/lockstep_cross_validator.go

Line 275 to 283 in be00aaa

```
275    // All chains must be ready and error-free
276    for _, ingester := range v.chains {
277        if ingester.Error() != nil {
278            return
279        }
280        if !ingester.Ready() {
281            return
282        }
283    }
```

For a detailed description of `CrossValidator` error handling, see `LogsDBChainIngester` informational findings.

Failure Recovery

Neither `CrossValidator` nor `ChainIngester` implement a way to clear their error states programmatically. (While `ChainIngester` does expose a `ClearError` method, it is only called in tests.) Restoring the service to a functioning state therefore requires manual intervention: stopping the service, restoring the database to a known-good state by either rewinding or flushing it, and then restarting the service.

We note that `AdminFrontend` does define a `Rewind` endpoint, but at the commit under investigation it is not yet implemented.

ethereum-optimism/optimism/op-interop-filter/filter/frontend.go

Line 49 to 54 in be00aaa

```
49  // Rewind is not implemented. For cross-chain consistency, rewind would need to
50  // coordinate across all chains to the same timestamp, which is complex.
51  // Instead, wipe the filter's data directory and restart fresh.
52  func (a *AdminFrontend) Rewind(ctx context.Context, chain eth.ChainID, block eth.BlockID) error {
53      return errors.New("rewind not implemented: wipe the filter's data directory and restart instead")
54  }
```

Normal Mode

When the service is not in failsafe mode, i.e. the manual failsafe is disabled, and neither `CrossValidator` nor any `ChainIngester` is in the error state, normal operation proceeds:

- `CheckAccessList` queries are served.
- The `CrossValidator` continues advancing the cross-validated timestamp.

- Each `ChainIngester` continues ingesting blocks into its `LogsDB`.

Both validation and ingestion are monotonic in the following sense:

- The cross-validated timestamp never decreases. On a validation error, the error state is set and further progress is blocked.
- The ingested chain is never rewound. On a detected reorg or ingestion error, the error state is set and further ingestion is blocked.

The ingester monotonicity guarantee is significant for correctness: once block `N` of a given L2 has been ingested, it permanently refers to the same block for the lifetime of the service, ruling out a class of race conditions between the ingestion and validation processes.

Differences to Supernode

Although the Interop Filter performs a similar cross-validation to the Supernode, there are a few notable distinctions:

- Unlike the Supernode, which fetches blocks from the L2 chains as needed during the cross-validation loop, the Interop Filter fetches L2 blocks asynchronously. While the `CrossValidator` component runs the cross-validation loop, each `ChainIngester` concurrently runs its own loop to retrieve L2 blocks and record them in its `LogsDB`, from where they are read by the `CrossValidator`.
- The L2 blocks processed by the Interop Filter are unsafe blocks transmitted via the gossip protocol, rather than safe blocks derived from L1.
- Cross-validation considers only executing messages that refer to an initiating message in the past, not messages at the same timestamp. As a consequence, no cycle check is required.
- Unlike the Supernode, which maintains a full `VerifiedDB`, the Interop Filter keeps track of only the latest cross-validated timestamp.
- The Interop Filter does not implement any rewind procedure. In the case of a reorg or a block failing cross-validation, the Interop Filter enters an error state, and recovery must be performed manually by deleting the data and restarting.

Fuzzing

This section presents an overview of the fuzzing methodology, a description of the fuzzing targets and random generators that compose the implementation, as well as a coverage report.

Unofficial

Preliminary Analysis

This section focuses on our preliminary findings when investigating how to implement the fuzzing harness. It was primarily written at the beginning of the engagement and mostly contains a breakdown of the components and interfaces in the Interop and an overview of candidates for mocking plus entrypoints for testing.

We cover the following:

- **Tested functions**: Which functions in each package cover interop verification while enabling efficient fuzzing through dependency isolation?
- **Test doubles**: Which dependencies should be isolated?

The following are out of scope for this section:

- **Properties**: Described in detail in Supernode Overview and Interop Filter Overview.
- **Test input generation**: The primary input for cross-chain verification is a view of the L1 and L2 chains. Fuzzing logic from previous engagements is expected to be largely reusable here.

Interop

Cross-safe verification is implemented through the `Interop` struct.

`Interop`

ethereum-optimism/optimism/op-supernode/supernode/activity/interop/interop.go

Line 39 to 67 in be00aaa

```go
// Interop is a VerificationActivity that can also run background work as a RunnableActivity.
type Interop struct {
  log                 log.Logger
  chains              map[eth.ChainID]cc.ChainContainer
  activationTimestamp uint64
  dataDir             string

  verifiedDB *VerifiedDB
  logsDBs    map[eth.ChainID]LogsDB

  mu      sync.RWMutex
  ctx     context.Context
  cancel  context.CancelFunc
  started bool

  currentL1 eth.BlockID

  verifyFn func(ts uint64, blocksAtTimestamp map[eth.ChainID]eth.BlockID) (Result, error)

  // cycleVerifyFn handles same-timestamp cycle verification.
  // It is called after verifyFn in progressInterop, and its results are merged.
  // Set to verifyCycleMessages by default in New().
  cycleVerifyFn func(ts uint64, blocksAtTimestamp map[eth.ChainID]eth.BlockID) (Result, error)

  // pauseAtTimestamp is used for integration test control only.
  // When non-zero, progressInterop will return early without processing
  // if the next timestamp to process is >= this value.
  pauseAtTimestamp atomic.Uint64
}
```

Tested Functions

We test three functions: `Interop.progressAndRecord`, `Interop.verifyInteropMessages`, and `Interop.verifyCycleMessages`.

`Interop.progressAndRecord`

implements a single round of interop validity checking.

We want to fuzz over this because it represents the largest 'unit' of validity checks and hosts all of the verification logic for doing so.

`Interop.verifyInteropMessages` and `Interop.verifyCycleMessages`

These are the default implementations of the `Interop.verifyFn` and `Interop.cycleVerifyFn` functions that the `Interop` struct uses, respectively, and are therefore also tested in isolation.

We choose to fuzz these in isolation because they should be more straightforward to implement and will act as stepping stones to developing the fuzzing harness over `progressAndRecord`.

`progressAndRecord`

ethereum-optimism/optimism/op-supernode/supernode/activity/interop/interop.go

Line 186 to 188 in be00aaa

```
186  // progressAndRecord attempts to progress interop and record the result.
187  // Returns (madeProgress, error) where madeProgress indicates if we advanced the verified timestamp.
188  func (i *Interop) progressAndRecord() (bool, error) {
```

`verifyInteropMessages`

ethereum-optimism/optimism/op-supernode/supernode/activity/interop/algo.go

Line 53 to 62 in be00aaa

```
53  // verifyInteropMessages validates all executing messages at the given timestamp.
54  // Returns a Result indicating whether all messages are valid or which chains have invalid blocks.
55  //
56  // For each chain:
57  // 1. Open the block from the logsDB and verify it matches blocksAtTimestamp
58  // 2. For each executing message in the block:
59  //    - Verify the initiating message exists in the source chain's logsDB
60  //    - Verify the initiating message timestamp <= executing message timestamp
61  //    - Verify the initiating message hasn't expired (within ExpiryTime)
62  func (i *Interop) verifyInteropMessages(ts uint64, blocksAtTimestamp blockPerChain) (Result, error) {
```

`verifyCycleMessages`

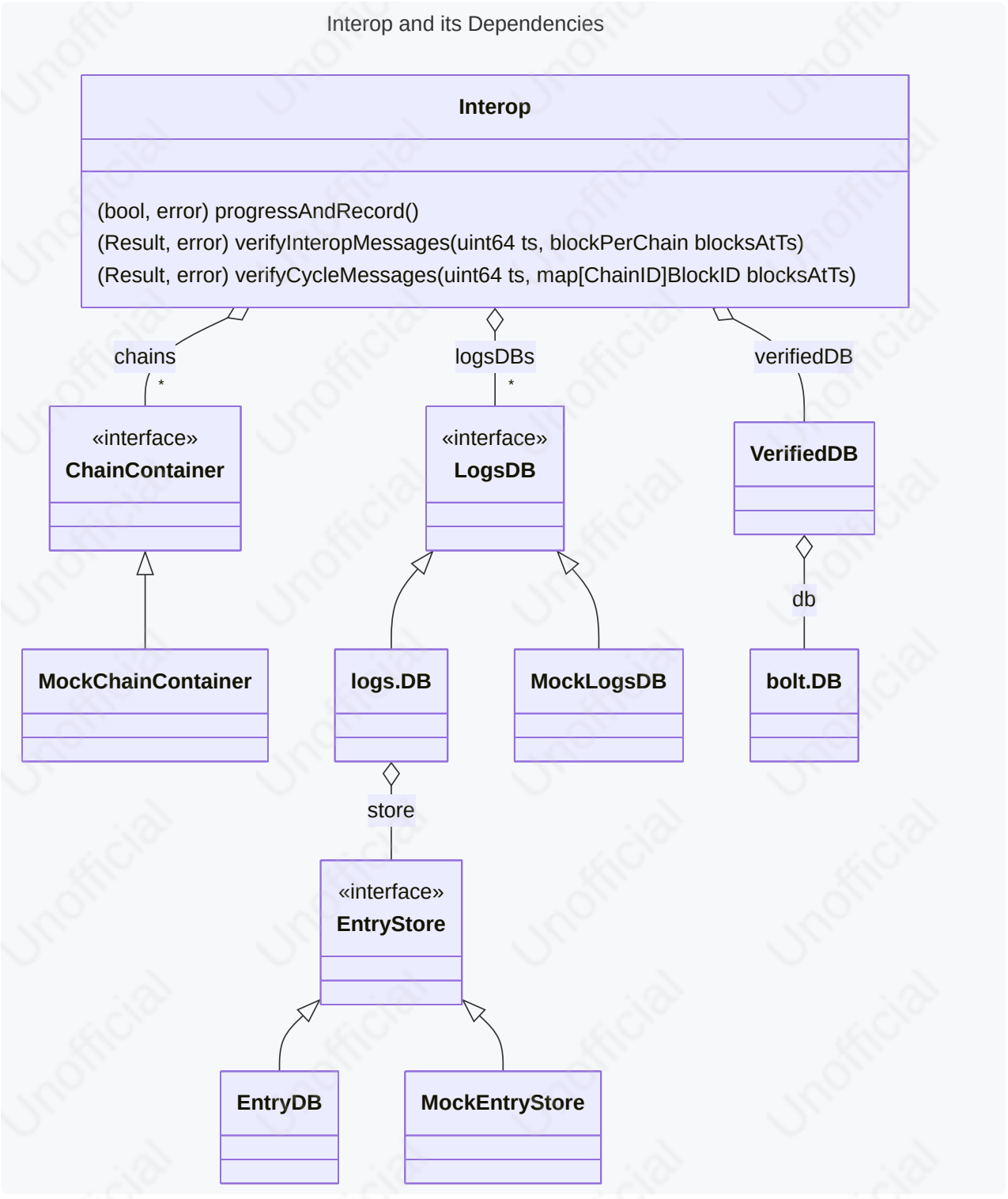
ethereum-optimism/optimism/op-supernode/supernode/activity/interop/cycle.go

Line 166 to 171 in 32bbf86

```
166  // verifyCycleMessages is the cycle verification function for same-timestamp interop.
167  // It verifies that same-timestamp executing messages form valid dependency relationships
168  // using Kahn's topological sort algorithm.
169  //
170  // Returns a Result with InvalidHeads populated for chains participating in cycles.
171  func (i *Interop) verifyCycleMessages(ts uint64, blocksAtTimestamp map[eth.ChainID]eth.BlockID)
(Result, error) {
```

Test Doubles

Setting up an `Interop` instance for testing requires three primary dependencies: `ChainContainer`, `LogsDB`, and `VerifiedDB`.


Interop and its Dependencies
Interop
(bool, error) progressAndRecord()
(Result, error) verifyInteropMessages(uint64 ts, blockPerChain blocksAtTs)
(Result, error) verifyCycleMessages(uint64 ts, map[ChainID]BlockID blocksAtTs)
chains
*
logsDBs
*
verifiedDB
«interface»
ChainContainer
«interface»
LogsDB
VerifiedDB
db
MockChainContainer
logs.DB
MockLogsDB
bolt.DB
store
«interface»
EntryStore
EntryDB
MockEntryStore

`ChainContainer`

Since `ChainContainer` is an interface, we can implement a test double to isolate this dependency. This is the recommended approach, as `simpleChainContainer`, which is the concrete implementation of `ChainContainer`, has its own complex dependencies (like `VirtualNode`, `EngineController`, `DenyList`, etc.).

ethereum-optimism/optimism/op-supernode/supernode/chain_container/chain_container.go

Line 32 to 63 in be00aaa

```
32  type ChainContainer interface {
33    Start(ctx context.Context) error
34    Stop(ctx context.Context) error
35    Pause(ctx context.Context) error
36    Resume(ctx context.Context) error
37
38    ID() eth.ChainID
39    LocalSafeBlockAtTimestamp(ctx context.Context, ts uint64) (eth.L2BlockRef, error)
40    SyncStatus(ctx context.Context) (*eth.SyncStatus, error)
41    VerifiedAt(ctx context.Context, ts uint64) (l2, l1 eth.BlockID, err error)
42    OptimisticAt(ctx context.Context, ts uint64) (l2, l1 eth.BlockID, err error)
43    OutputRootAtL2BlockNumber(ctx context.Context, l2BlockNum uint64) (eth.Bytes32, error)
44    OptimisticOutputAtTimestamp(ctx context.Context, ts uint64) (*eth.OutputResponse, error)
45    // RewindEngine rewinds the engine to the highest block with timestamp less than or equal to the
given timestamp.
46    // invalidatedBlock is the block that triggered the rewind and is passed to reset callbacks.
47    RewindEngine(ctx context.Context, timestamp uint64, invalidatedBlock eth.BlockRef) error
48    RegisterVerifier(v activity.VerificationActivity)
49    // FetchReceipts fetches the receipts for a given block by hash.
50    // Returns block info and receipts, or an error if the block or receipts cannot be fetched.
51    FetchReceipts(ctx context.Context, blockHash eth.BlockID) (eth.BlockInfo, types.Receipts, error)
52    // BlockTime returns the block time in seconds for this chain.
53    BlockTime() uint64
54    // InvalidateBlock adds a block to the deny list and triggers a rewind if the chain
55    // currently uses that block at the specified height.
56    // Returns true if a rewind was triggered, false otherwise.
57    InvalidateBlock(ctx context.Context, height uint64, payloadHash common.Hash) (bool, error)
58    // IsDenied checks if a block hash is on the deny list at the given height.
59    IsDenied(height uint64, payloadHash common.Hash) (bool, error)
60    // SetResetCallback sets a callback that is invoked when the chain resets.
61    // The supernode uses this to notify activities about chain resets.
62    SetResetCallback(cb ResetCallback)
63  }
```

LogsDB

Since `LogsDB` is an interface, we can implement a test double to isolate this dependency. However, two other approaches are also feasible: mocking a simpler, lower-level interface (`EntryStore`), or using the actual dependency without mocking (`EntryDB`). In the latter case, the test harness should provide an in-memory file for the underlying `EntryDB` .

ethereum-optimism/optimism/op-supernode/supernode/activity/interop/logdb.go

Line 20 to 44 in be00aaa

```
20  // LogsDB is the interface for interacting with a chain's logs database.
21  // *logs.DB implements this interface.
22  type LogsDB interface {
23   // LatestSealedBlock returns the latest sealed block ID, or false if no blocks are sealed.
24   LatestSealedBlock() (eth.BlockID, bool)
25   // FirstSealedBlock returns the first block seal in the DB.
26   FirstSealedBlock() (types.BlockSeal, error)
27   // FindSealedBlock returns the block seal for the given block number.
28   FindSealedBlock(number uint64) (types.BlockSeal, error)
29   // OpenBlock returns the block reference, log count, and executing messages for a block.
30   OpenBlock(blockNum uint64) (ref eth.BlockRef, logCount uint32, execMsgs
map[uint32]*types.ExecutingMessage, err error)
31   // Contains checks if an initiating message exists in the database.
32   // Returns the block seal if found, or an error (ErrConflict if not found, ErrFuture if not yet
indexed).
33   Contains(query types.ContainsQuery) (types.BlockSeal, error)
34   // AddLog adds a log entry to the database.
35   AddLog(logHash common.Hash, parentBlock eth.BlockID, logIdx uint32, execMsg
*types.ExecutingMessage) error
36   // SealBlock seals a block in the database.
37   SealBlock(parentHash common.Hash, block eth.BlockID, timestamp uint64) error
38   // Rewind removes all blocks after newHead from the database.
39   Rewind(inv reads.Invalidator, newHead eth.BlockID) error
40   // Clear removes all data from the database.
41   Clear(inv reads.Invalidator) error
42   // Close closes the database.
43   Close() error
44  }
```

ethereum-optimism/optimism/op-supervisor/supervisor/backend/db/logs/db.go

Line 34 to 50 in be00aaa

```
34  // DB implements an append only database for log data and cross-chain dependencies.
35  //
36  // To keep the append-only format, reduce data size, and support reorg detection and registering of
executing-messages:
37  //
38  // Use a fixed 24 bytes per entry.
39  //
40  // Data is an append-only log, that can be binary searched for any necessary event data.
41  type DB struct {
42      log    log.Logger
43      m      Metrics
44      store  entrydb.EntryStore[EntryType, Entry]
45      rwLock sync.RWMutex
46
47      chainID eth.ChainID
48
49      lastEntryContext logContext
50  }
```

ethereum-optimism/optimism/op-supervisor/supervisor/backend/db/entrydb/entry_db.go

Line 12 to 19 in be00aaa

```
12  type EntryStore[T EntryType, E Entry[T]] interface {
13      Size() int64
14      LastEntryIdx() EntryIdx
15      Read(idx EntryIdx) (E, error)
16      Append(entries ...E) error
17      Truncate(idx EntryIdx) error
18      Close() error
19  }
```

ethereum-optimism/optimism/op-supervisor/supervisor/backend/db/entrydb/entry_db.go

Line 51 to 58 in be00aaa

```
51  type EntryDB[T EntryType, E Entry[T], B Binary[T, E]] struct {
52      data         dataAccess
53      lastEntryIdx EntryIdx
54
55      b B
56
57      cleanupFailedWrite bool
58  }
```

`VerifiedDB`

`VerifiedDB` is a concrete struct and thus should be instantiated directly during test setup. For efficiency, the test harness should provide an in-memory file for the underlying `bolt.DB`.

ethereum-optimism/optimism/op-supernode/supernode/activity/interop/verified_db.go

Line 28 to 32 in be00aaa

```
28  type VerifiedDB struct {
29      db            *bolt.DB
30      lastTimestamp uint64
31      initialized   bool
32  }
```

Interop Filter

The `LockstepCrossValidator` struct is the default implementation of the `CrossValidator` interface, providing cross-unsafe verification.

`LockstepCrossValidator`

ethereum-optimism/optimism/op-interop-filter/filter/lockstep_cross_validator.go

Line 17 to 52 in be00aaa

```go
// LockstepCrossValidator validates cross-chain executing messages and tracks
// the cross-validated timestamp.
//
// "Lockstep" refers to its synchronization model: all chains must reach the same
// timestamp before validation can advance. This is simpler but means a slow chain
// holds back validation for all chains.
//
// Simplifications in this implementation:
//   - No cycle detection: same-block executing messages are not supported
//   - Lockstep advancement: waits for ALL chains to reach timestamp T before
//     validating T, rather than validating each chain independently
//
// Future improvement: per-chain validation that tracks cross-validated timestamp
// independently for each chain, allowing faster chains to advance without waiting.
type LockstepCrossValidator struct {
    log     log.Logger
    metrics metrics.Metricer

    messageExpiryWindow uint64
    startTimestamp      uint64 // Initial timestamp to start validation from
    validationInterval  time.Duration

    // Chain ingesters keyed by chain ID (read-only after construction)
    chains map[eth.ChainID]ChainIngester

    // Single global cross-validated timestamp
    crossValidatedTs atomic.Uint64

    // Error state for validation failures
    errMu sync.RWMutex
    err   *ValidatorError

    ctx    context.Context
    cancel context.CancelFunc
    wg     sync.WaitGroup
}
```

`CrossValidator`

ethereum-optimism/optimism/op-interop-filter/filter/interfaces.go

Line 53 to 72 in be00aaa

```
53  // CrossValidator validates cross-chain messages.
54  // Implementations:
55  //   - LockstepCrossValidator: waits for all chains to align before advancing
56  type CrossValidator interface {
57    // Start begins the validator's background processing.
58    Start() error
59
60    // Stop halts the validator's background processing.
61    Stop() error
62
63    // ValidateAccessEntry validates a single access list entry.
64    ValidateAccessEntry(access types.Access, minSafety types.SafetyLevel, execDescriptor
types.ExecutingDescriptor) error
65
66    // CrossValidatedTimestamp returns the global cross-validated timestamp.
67    CrossValidatedTimestamp() (uint64, bool)
68
69    // Error returns the current error state, if any.
70    // Validation errors (invalid executing messages) are tracked here.
71    Error() *ValidatorError
72  }
```

Tested Functions

We test two functions: `LockstepCrossValidator.advanceValidation` and `LockstepCrossValidator.ValidateAccessEntry`, the latter of which is defined in the `CrossValidator` interface.

`advanceValidation`

ethereum-optimism/optimism/op-interop-filter/filter/lockstep_cross_validator.go

Line 268 to 269 in be00aaa

```
268  // advanceValidation tries to advance the cross-validated timestamp one step at a time.
269  func (v *LockstepCrossValidator) advanceValidation() {
```

`ValidateAccessEntry`

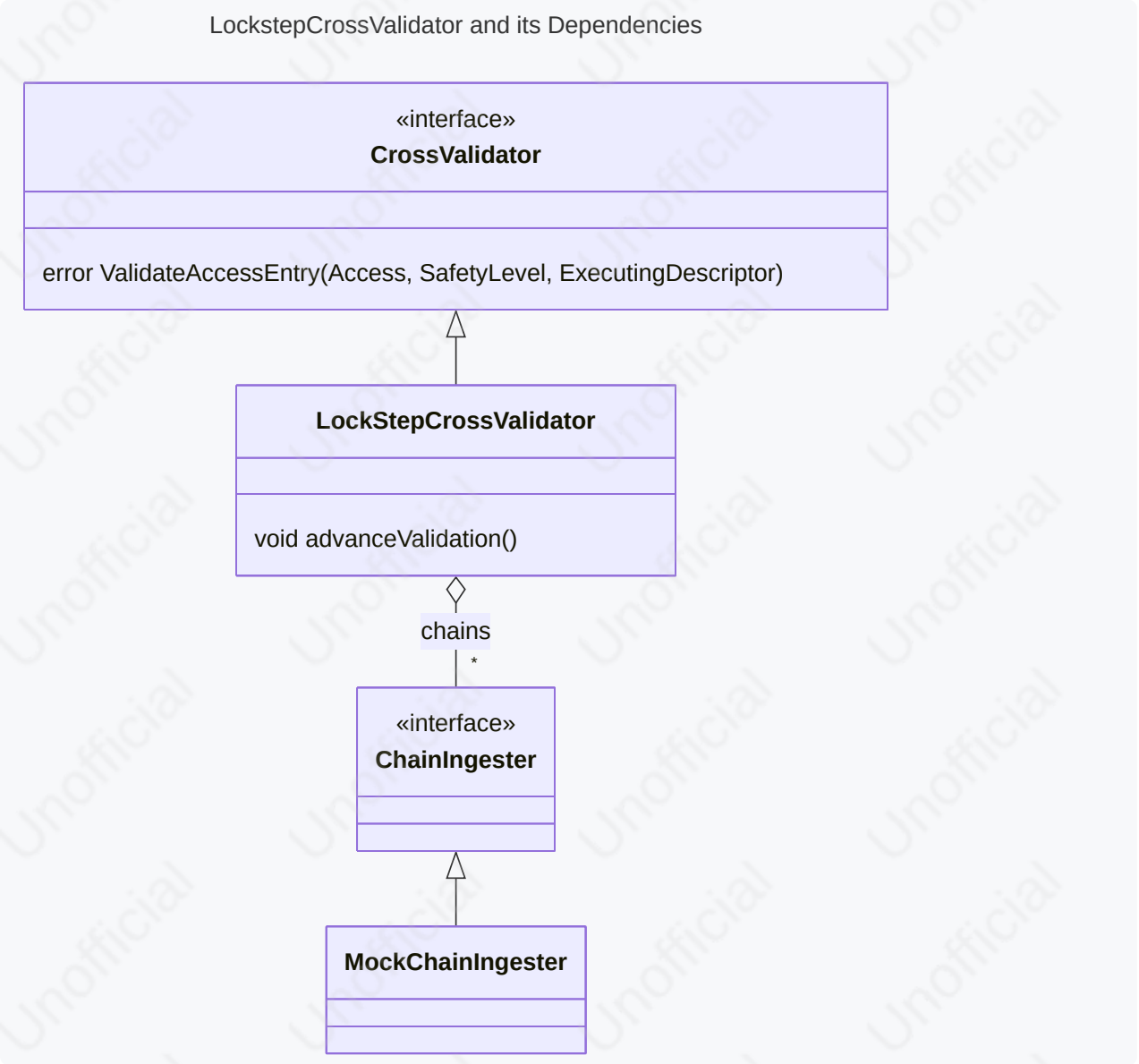
ethereum-optimism/optimism/op-interop-filter/filter/lockstep_cross_validator.go

Line 170 to 171 in be00aaa

```
170  // ValidateAccessEntry validates a single access list entry against all message validity rules.
171  func (v *LockstepCrossValidator) ValidateAccessEntry(
```

Test Doubles

`LockstepCrossValidator` has only one significant dependency: `ChainIngester`.



`ChainIngester`

Since `ChainIngester` is an interface, we can implement a test double to isolate this dependency. This is the recommended approach, as its default implementation, `LogsDBChainIngester`, depends on both a DB and an RPC client.

ethereum-optimism/optimism/op-interop-filter/filter/interfaces.go

Line 17 to 51 in be00aaa

```go
// ChainIngester provides access to chain logs and state.
// Implementations include:
//   - mockChainIngester: in-memory for testing
//   - LogsDBChainIngester: RPC-backed with logsdb for production
type ChainIngester interface {
  // Start begins the ingester's background processing.
  Start() error

  // Stop halts the ingester's background processing.
  Stop() error

  // Contains checks if a log exists in the chain's database.
  Contains(query types.ContainsQuery) (types.BlockSeal, error)

  // LatestBlock returns the latest ingested block.
  LatestBlock() (eth.BlockID, bool)

  // LatestTimestamp returns the timestamp of the latest ingested block.
  LatestTimestamp() (uint64, bool)

  // GetExecMsgsAtTimestamp returns executing messages with the given inclusion timestamp.
  GetExecMsgsAtTimestamp(timestamp uint64) ([]IncludedMessage, error)

  // Ready returns true if the ingester has completed initial sync.
  Ready() bool

  // Error returns the current error state, if any.
  Error() *IngesterError

  // SetError sets an error state on the ingester.
  SetError(reason IngesterErrorReason, msg string)

  // ClearError clears the error state.
  ClearError()
}
```

Fuzzing Implementation

Methodology

The entrypoints under test were `Interop.progressAndRecord`, `Interop.verifyInteropMessages`, and `Interop.verifyCycleMessages`.

Work was primarily done on improving the input generation for the tests. The chain randomizer from the previous engagement was brought in and adapted to the new interface as well as refactored and debugged.

The fuzz target first generates the random L2 chains, and uses `progressAndRecord` to drive the population of the logsDB. It then runs `verifyInteropMessages` against the set of most current blocks in the random chains and makes assertions about the result based on how the chain was generated (ie. if the chain is expected to be invalid, then there should be an error)

ChainRandomizer improvements

`Implementing ChainContainer`

The interop uses an interface called `ChainContainer` as its source for L2 chain information. The implementation it uses has complex dependencies. But, this interface affords us a great opportunity to make our own implementation for tests.

A new RandomChainContainer type was introduced that implements the parts of the ChainContainer interface that are required for our tests, and stubs the unused/inapplicable methods. The L2 chain data supplied by it is backed by the randomized data coming from the chain randomizer. This allows the fuzzer to plug generated chains directly into the real interop verification logic rather than relying on mock processor sources.

`Block time awareness`

The original randomizer generated blocks with random timestamps and then assigned them to chains, with no concept of per-chain block times. This presents an issue with the new interop because it is much more strict about the cadence of blocks arriving on the chains.

The newest version of the randomizer now assigns each chain a random blockTime and uses a NextValidChain scheduling function that always selects the chain whose next block would have a valid timestamp. This ensures the generated chains have valid timestamps from the start of each chain to the end.

`Removal of the global cutoff / head-tracking model`

The previous version maintained global cutoff indices for cross-unsafe, cross-safe, local-unsafe, and local-safe heads and computed per-chain heads from those cutoffs. This entire mechanism (ChainHeads, cutoffs, GetCrossUnsafeCandidate, GetCrossSafeCandidate) was removed. The new randomizer focuses on generating structurally valid chains and defers head-tracking to invalidation routines.

`Conversion to member methods`

Free functions like addRandomInitiatingMessage, addExecutingMessage, GenerateReceiptsFromLogs, and the invalidation functions (InsertCycle, InsertSelfDependency, etc.) were converted into methods on RandomChain. This improves readability and removes the need to thread the RandomChain pointer and rand.Rand through every call.

`Simplified cycle generation`

The original InsertCycle relied on a recursive hazard-set traversal (listHazards) to find blocks with the same timestamp reachable through dependencies, then picked start/end points within those hazards. The new CreateCycle uses a simpler SameTimeStampSets utility that groups contiguous blocks sharing a timestamp, picks a set, and creates a circular chain of dependencies through all members of the group.

`Inline invalidation with probability`

Rather than invalidating a specific candidate after the fact, the new code uses an invalidateChance parameter and calls Invalidate() at the end of chain construction, choosing randomly among cycle, self-dependency, invalid identifier, and future dependency. The chain will then have an `isInvalid` bit set for the tests to make proper assertions about whether to expect an error or not.

Fuzz Target Overview

The following is a description of the fuzz target we made to test against Interop.

The target takes two integers as random parameters. The first is a seed which drives the randomization of input generation. The second is the number of L2 chains to generate, bit masked to cap the value at 8 (and a minimum of 2 chains are generated).

The function parameters along with some other hardcoded limits on the random chain generator are used to instantiate a `interopFuzzHarness` struct, which is modeled after a `interopTestHarness` struct that exists in the unit tests. This seeds the random chain generator and creates the chains and their corresponding ChainContainers. The Interop gets instantiated with the ChainContainers as its source for L2 data.

The LogsDB for the Interop needs to be populated. We rely on its `progressAndRecord` method to drive forward the current timestamp. The method returns a boolean indicating if the timestamp was successfully updated or not, so we call it in a loop until it returns false. By this point, the timestamp should be at the

latest timestamp of all the blocks in the randomly generated chains if they are expected to be valid, otherwise it will be at the timestamp where the first invalidity occurs.

To actually test the validity of the chain at this timestamp, we use the Interop's `checkChainsReady` to get the current block for each chain at that timestamp.

Finally, verifyInteropMessages is called with the timestamp and the blocks. This returns a result and a possible error. If our randomly generated chain is expected to be valid, we make an assertion that the result is valid and that there was no error.

`requireLogsDBChainIntegrity`

Walks every LogsDB and checks that the sealed blocks form a valid chain. For each consecutive pair of blocks it asserts:

- Block numbers are sequential (`prev.Number + 1 == curr.Number`).
- The parent hash of the current block equals the hash of the previous block (`curr.ParentHash == prev.Hash`).
- Timestamps advance by exactly blockTime (`curr.Timestamp == prev.Timestamp + blockTime`).

`requireVerifiedDBChainIntegrity`

Operates on the VerifiedDB and checks two invariants:

- Monotonic head advancement: for every consecutive pair of timestamps in the VerifiedDB, each chain's L2 head is either identical (the chain did not produce a block at that timestamp) or a direct successor (block number increases by exactly 1). A head must never skip a block or move backwards.
- LogsDB anchoring: the L2 heads recorded at both the activation timestamp and the last verified timestamp must exist in the corresponding LogsDB with the same block number and block hash. This ensures the VerifiedDB and LogsDB remain consistent at the boundaries of the verified range.

Chain Randomizer Overview

Our randomizer attempts to create multiple different scenarios of invalidity on the chains. Invalid cases we create include:

- Cyclic dependencies
- Self dependencies
- Invalid executing messages
- Future dependencies

These cases are scoped strictly on data coming from the L2 chains. Errors that are more operative in nature aren't covered and include things like:

- Missing L1 inclusions
- Missing LogsDBs
- Block mismatches between LogsDBs and ChainContainer data

Coverage Report

A breakdown by file of statement coverage in the interop package:

File	Coverage
algo.go	71.2%
checker.go	50.0%
cycle.go	88.6%
interop.go	43.9%
logdb.go	74.7%
types.go	100%
verification_view.go	93.8%
verified_db.go	54.4%

Overall coverage is 58.4%

The poor coverage over `interop.go` can be explained by much of it containing system level logic like starting/stopping/waiting and also database rewinding logic which isn't tested by our harness. If we ignore interop.go, then overall coverage goes up to 70.4%

Findings

This section contains all issues identified during the audit that could lead to unintended behavior, security vulnerabilities, or failure to enforce the protocol's intended logic. Each issue is documented with a description, potential impact, and recommended remediation steps.

Unofficial

Supernode Findings

The following are findings related to the code of the Supernode, found under the `op-supernode` subdirectory of the OP monorepo.

`earliestL1Inclusion` calculated by taking lowest block instead of highest

Severity: High | Difficulty: Low | Recommended Action: Fix Code | Addressed by client

The `verifyInteropMessages` function (here) starts by calculating the `earliestL1Inclusion`, which is the earliest L1 block where all L2 blocks for the given timestamp are available. This is equivalent to the *highest* among the L1 blocks from which each L2 block was derived. However, currently the implementation calculates the `earliestL1Inclusion` as the *lowest* among the L1 blocks: it starts with `earliestL1Inclusion.Number = MaxUint64` (here) and then updates it whenever `l1Block.Number < earliestL1Inclusion.Number` (here). Instead, it should start as 0 and update it whenever `earliestL1Inclusion.Number < l1Block.Number`.

Status

Fixed in PR #19343.

`BlockNumberToTimestamp` duplication and inconsistencies

Severity: Low | Difficulty: High | Recommended Action: Fix Code | Addressed by client

`chain_container.go` includes two functions called `BlockNumberToTimestamp` and `blockNumberToTimestamp`:

runtimeverification/_audits_Ethereum-optimism_optimism_interopv2/op-supernode/supernode/chain_container/chain_container.go

Line 296 to 301 in be00aaa

```
296 func (c *simpleChainContainer) BlockNumberToTimestamp(ctx context.Context, blocknum uint64) (uint64,
error) {
297  if c.vncfg == nil {
298      return 0, fmt.Errorf("rollup config not available")
299  }
300  return c.vncfg.Rollup.Genesis.L2Time + (blocknum * c.vncfg.Rollup.BlockTime), nil
301 }
```

runtimeverification/_audits_Ethereum-optimism_optimism_interopv2/op-supernode/supernode/chain_container/chain_container.go

Line 547 to 553 in be00aaa

```
547 // blockNumberToTimestamp converts a block number to its timestamp using rollup config.
548 func (c *simpleChainContainer) blockNumberToTimestamp(blockNum uint64) uint64 {
549  if c.vncfg == nil {
550      return 0
551  }
552  return c.vncfg.Rollup.Genesis.L2Time + (blockNum * c.vncfg.Rollup.BlockTime)
553 }
```

- These functions are a duplication of each other, except that `blockNumberToTimestamp` doesn't return an error and just returns 0 if the virtual node config is empty.
- Their calculation seems to assume that the genesis block timestamp is always 0. This is inconsistent with `c.vncfg.Rollup.TargetBlockNumber()`, which subtracts `cfg.Genesis.L2Time` to calculate the result.

Status

Fixed in PR #19542.

`SyncStatus()` can silently fail and return empty result

Severity: High | Difficulty: Medium | Recommended Action: Fix Code | Addressed by client

The `SyncStatus()` function on `chain_container.go` can return an empty `SyncStatus{}` result, both directly in line 335 and via the call to `c.vn.SyncStatus()` in line 337:

runtimeverification/_audits_Ethereum-optimism_optimism_interopv2/op-supernode/supernode/chain_container/chain_container.go

Line 330 to 342 in be00aaa

```
330 func (c *simpleChainContainer) SyncStatus(ctx context.Context) (*eth.SyncStatus, error) {
331   if c.vn == nil {
332       if c.log != nil {
333           c.log.Warn("SyncStatus: virtual node not initialized")
334       }
335       return &eth.SyncStatus{}, nil
336   }
337   st, err := c.vn.SyncStatus(ctx)
338   if err != nil {
339       return nil, err
340   }
341   return st, nil
342 }
```

The function is called in a few places in the codebase, but they don't seem to check for the case when the return value is empty. This could cause the exceptional empty value to be processed like a valid status result.

Status

Fixed in PR #19543.

Supernode missing consistency checks to detect and recover from reorgs

Severity: High | Difficulty: Medium | Recommended Action: Fix Design | Addressed by client

The Supernode does not currently implement any kind of consistency check to detect when the L1 or L2 chains have suffered a reorg and to recover from it. This means that if a reorg happens, it might prevent the Supernode from advancing due to the new L2 blocks being inconsistent with the previous L2 blocks in the logs database, or in the worst case place the Supernode in an invalid state where new verified entries are not in the same chain as previous verified entries.

Note that due to asynchrony, reorgs could happen at any time, including in the middle of a round of cross-validation. Additionally, L2 chains might take time to sync up to an L1 reorg. Therefore, multiple queries to the L1 and L2 chains during a single round of cross-validation might return inconsistent results. It's important to ensure that the data fetched during a single iteration is a self-consistent snapshot of the chain states, besides being consistent with previous rounds of cross-validation.

Recommendation

Add consistency checks in the cross-validation loop of the Supernode ensuring that:

1. The L2 blocks fetched for the current timestamp are all consistent with each other; that is, they were all derived from L1 blocks forming a single linear chain.
2. They are all consistent with the previous verified set of L2 blocks; that is, the current linear chain is a continuation of the previously-verified L1 inclusion.

When (1) is violated, this means that the L2 chains are in the middle of a reorg and we need to wait until they have all synced up to the new L1. When (2) is violated, it means that a reorg has invalidated the previous set of verified blocks, and the state of the Supernode needs to be rewound to the previous timestamp.

Status

An updated Supernode with consistency checks implemented has been provided in commit `bd29b7d` . All of the subsequent Supernode findings are based on this updated commit. Note that there are pending follow-up items that are summarized in interop: follow-up items from strong-consistency review.

Operations that modify logs DB should leave contents untouched in case of failure

Severity: High | Difficulty: Medium | Recommended Action: Fix Code | Not addressed by client

When an operation that modifies the logs DB fails, the contents of the database should remain unchanged to prevent corruption. However, it's unclear if the current implementations are always safe in this respect. For example, `Rewind` calls `Truncate` and then `init`, and if the call to `init` returns an error, the state changes from `Truncate` are still persisted:

runtimeverification/_audits_Ethereum-optimism_optimism_interopv2/op-supervisor/supervisor/backend/db/logs/db.go

Line 641 to 649 in bd29b7d

```
641    // Truncate to contain idx entries. The Truncate func keeps the given index as last index.
642    if err := db.store.Truncate(iter.NextIndex() - 1); err != nil {
643        return fmt.Errorf("failed to truncate to block %s: %w", newHead, err)
644    }
645    // Use db.init() to find the log context for the new latest log entry
646    if err := db.init(true); err != nil {
647        return fmt.Errorf("failed to find new last entry context: %w", err)
648    }
649    return nil
```

If `init` can actually fail at this point, the database can be left in an invalid state that might be difficult to recover from.

Rewind is triggered even if previous verified L1 is still valid

Severity: Medium | Difficulty: Medium | Recommended Action: Fix Design | Not addressed by client

The `observeRound()` function in `interop.go` calls `SameL1Chain()`, and this validation returns a `bool` that is stored in `obs.L1Consistent`:

runtimeverification/_audits_Ethereum-optimism_optimism_interopv2/op-supernode/supernode/activity/interop/interop.go

Line 370 to 374 in bd29b7d

```
370         same, err := i.l1Checker.SameL1Chain(i.ctx, heads)
371         if err != nil {
372             return obs, fmt.Errorf("L1 consistency check: %w", err)
373         }
374         obs.L1Consistent = same
```

When `obs.L1Consistent == false`, this causes `checkPreconditions()` to return `DecisionRewind`, which will trigger the rewind procedure of the Supernode:

runtimeverification/_audits_Ethereum-optimism_optimism_interopv2/op-supernode/supernode/activity/interop/interop.go

Line 239 to 242 in bd29b7d

```
239     if !obs.L1Consistent {
240         output := StepOutput{Decision: DecisionRewind}
241         return &output
242     }
```

However, there are two separate reasons why `SameL1Chain()` might return `false`: if the previous verified L1 head in the `VerifiedDB` is no longer valid, or if any of the L2 heads are derived from an L1 that is no longer valid. Only the former case strictly necessitates a rewind, as the latter case might not mean that the previous verified L1 is no longer valid. The latter case likely means that an L2 has not caught up to an L1 reorg yet, and therefore we should wait until it syncs up.

Recommendation

Differentiate between the two cases during `progressInterop()`, so that the first case results in `DecisionRewind` and the second in `DecisionWait`.

Unnecessary rewind of the executing engine in `DecisionRewind` case

Severity: Medium | Difficulty: Medium | Recommended Action: Fix Design | Not addressed by client

When `progressInterop()` returns a `DecisionRewind`, this indicates that the state of the Supernode needs to be rewound due to an invalid L1 head. In this case, `applyRewindPlan()` is called to perform this rewind. In addition to rewinding the `VerifiedDB`, `LogsDB` and `DenyList`, `applyRewindPlan()` also calls `RewindEngine()` on the `ChainContainer`:

runtimeverification/_audits_Ethereum-optimism_optimism_interopv2/op-supernode/supernode/activity/interop/interop.go

Line 560 to 565 in bd29b7d

```
560         if plan.ResetAllChainsTo != nil {
561             if err := chain.RewindEngine(i.ctx, *plan.ResetAllChainsTo, eth.BlockRef{}); err != nil {
562                 i.log.Error("failed to reset chain engine on rewind", "chain", chainID, "err", err)
563                 recordErr(fmt.Errorf("chain %s: reset chain engine on rewind: %w", chainID, err))
564             }
565         }
```

This call rewinds the executing engine via the `EngineController` to the previous timestamp. However, this is not actually necessary in this case, since the goal is to rewind the state of the Supernode to align with the current state of the chains, not the other way around. In fact, the `DecisionRewind` case indicates that the chains have suffered or are suffering a reorg, and it might be preferable to just keep re-trying the cross-validation loop until they have settled into a consistent state (and the Supernode has been rewound far back enough to be consistent with the current chain state).

Additionally, rewinding the engine in the `DecisionRewind` case brings up the following issues:

- If `applyRewindPlan()` rewinds the engine but still fails because of an error when rewinding the `LogsDB`, the rewind will remain as a pending transition and re-attempted on the next iteration. In this case, the engine will be rewound again to the same block, repeating the procedure which involves two forkchoice updates (one to a synthetic block, and another back to the actual target block). Additionally, if there is the possibility that the chain has advanced beyond that block, or suffered a reorg to a different fork, it will be brought back unnecessarily.
- As can be seen above, `RewindEngine()` is not actually called when `plan.ResetAllChainsTo == nil`. This happens in the case when the rewind would completely clear the `VerifiedDB` and `LogsDB` (and rewind back to before the activation timestamp), and is because if `plan.ResetAllChainsTo == nil` we lack information of what timestamp to rewind the engine back

to. However, this is inconsistent: if we want to rewind the engine on `DecisionRewind`, it would be expected to also happen in this case. Avoiding rewinding the engine entirely would avoid this inconsistency.

Inaccurate debug message in `verifyCanAddTimestamp()` when `seal.Timestamp == ts`

Severity: Low | Difficulty: Medium | Recommended Action: Fix Code | Not addressed by client

`verifyCanAddTimestamp()` in `logsdb.go` returns early if the timestamp of the last sealed block in the `LogsDB` is greater than the timestamp of the block we're currently trying to insert:

runtimeverification/_audits_Ethereum-optimism_optimism_interopv2/op-supernode/supernode/activity/interop/logdb.go

Line 169 to 178 in bd29b7d

```
169    // determine the timestamp of the last sealed block
170    seal, err := db.FindSealedBlock(latestBlock.Number)
171    if err != nil {
172        return eth.BlockID{}, hasBlocks, fmt.Errorf("chain %s: failed to find sealed block %d: %w",
chainID, latestBlock.Number, err)
173    }
174
175    // if the last sealed block is already after the timestamp in question, return success
176    if seal.Timestamp > ts {
177        return latestBlock, hasBlocks, nil
178    }
```

Consider changing `seal.Timestamp > ts` to `seal.Timestamp >= ts`. The case when the two are equal might happen if we are trying to re-execute a pending transition that failed after the block was inserted. In this case, `gap := ts - seal.Timestamp` will be calculated as 0, which will produce the following debug message:

runtimeverification/_audits_Ethereum-optimism_optimism_interopv2/op-supernode/supernode/activity/interop/logdb.go

Line 188 to 198 in bd29b7d

```
188    // If the gap is less than a block time, we can still append the timestamp to the database.
189    // This is expected for chains whose block time is greater than one second, since the
190    // interop timestamp may legitimately fall between consecutive L2 blocks.
191    if gap < blockTime {
192        i.log.Debug("verifyCanAddTimestamp: timestamp falls between L2 blocks for this chain; this can be expected for chains with block times greater than one second",
193            "chain", chainID,
194            "timestamp", ts,
195            "block time", blockTime,
196            "gap", gap,
197        )
198    }
```

Although this doesn't change the result of the timestamp verification, the debug message is not appropriate for this case, since the gap being 0 is independent of the block time and simply a product of trying to re-insert the same block. Returning early in the `seal.Timestamp == ts` prevents this.

Construction of synthetic block might fail if block execution depends on fee recipient

Severity: High | Difficulty: Low | Recommended Action: Fix Code | Not addressed by client

`RewindToTimestamp()` in `rewind.go` temporarily performs an FCU to a synthetic block before doing an FCU back to the actual target block. This synthetic block is constructed to have the same block number, timestamp, content and parent as the target block, but its fee recipient is modified in order to produce a different hash:

runtimeverification/_audits_Ethereum-optimism_optimism_interopv2/op-supernode/supernode/chain_container/engine_controller/rewind.go

Line 47 to 52 in bd29b7d

```
47      // Step 1: Insert a synthetic block (modified fee recipient) which
48      // is built on the parent of the target block:
49      // [n-1,parent] <-- [n,target] <--...<-- [m>n,unsafe]
50      //
51      //                 [n,synthetic]
52      syntheticBlockHash, err := e.insertSyntheticPayload(ctx, targetBlock.Number)
```

However, the fee recipient of a block can influence the execution of the transactions within the block (for example, it is observable in EVM code using the `COINBASE` opcode). As a result, if the block contains a transaction that depends on the fee recipient, modifying the fee recipient without recomputing the payload's state root and other execution-derived fields might cause it to be rejected when `NewPayload()` is called from within `insertSyntheticPayload()`. This would cause an error and make it impossible to go through with the rewind to the target block no matter how many times it is attempted, potentially halting progress of cross-validation.

Recommendation

Construct the synthetic block by modifying a field that does not affect execution results, such as `ExtraData`, instead of `FeeRecipient`. Alternatively, provide a fallback that recomputes fields that depend on execution results, such as the state root, in the case that `NewPayload()` fails.

Block invalidation may fail silently without rewinding executing engine

Severity: Medium | Difficulty: Medium | Recommended Action: Fix Code | Not addressed by client

`InvalidateBlock()` in `invalidation.go` returns `false` with no error if it fails to retrieve the block from the `EngineController`, which happens in the following cases:

1. When the `EngineController` is `nil`.
2. When the `EngineController` fails to retrieve an `L2BlockRef` for the block (via `L2BlockRefByNumber()`).
3. When the hash of the retrieved `L2BlockRef` doesn't match the block.

In any of these cases, the invalidation procedure will continue silently with no error, but the execution engine will not be rewound to the previous block. This could happen, for example, if `L2BlockRefByNumber()` fails due to a network problem, leaving the execution layer out of sync with the Supernode.

Note that `invalidateBlock()` in `interop.go` ignores the boolean return value of `InvalidateBlock()` and returns only the error, so the fact that it returns `false` in this case doesn't affect the outcome.

`l1Inclusion()` might retrieve inconsistent L1 block in the case of reorg

Severity: Medium | Difficulty: High | Recommended Action: Fix Code | Not addressed by client

The function `l1Inclusion()` in `algo.go` calls `OptimisticAt()` on the `ChainContainer` to obtain the L1 block that each L2 head was derived from:

runtimeverification/_audits_Ethereum-optimism_optimism_interopv2/op-supernode/supernode/activity/interop/algo.go

Line 41 to 45 in bd29b7d

```
41        _, l1Block, err := chain.OptimisticAt(i.ctx, ts)
42        if err != nil {
43            i.log.Error("failed to get L1 inclusion for L2 block", "chainID", chainID, "timestamp", ts,
"err", err)
44            return eth.BlockID{}, fmt.Errorf("chain %s: failed to get L1 inclusion: %w", chainID, err)
45        }
```

However, if there has been a reorg between fetching the L2 blocks and calling this function, this information might be incorrect, as the L2 block at the given timestamp might have changed. Instead, the L1 inclusion should be calculated using the L1 heads already retrieved previously by `observeRound()`.

Interop Filter Findings

The following are findings related to the code of the Interop Filter, found under the `op-interop-filter` subdirectory of the OP monorepo.

Unofficial

`LockstepCrossValidator` never advances if `startTimestamp` is `0`

Severity: Low | Difficulty: Low | Recommended Action: Fix Code | Addressed by client

In `advanceValidation`, value `0` is used to mark that `v.crossValidatedTs` is uninitialized. If it's uninitialized, `v.startTimestamp` is written back to `v.crossValidatedTs`, then the function returns. If `v.startTimestamp` is `0` (which is not checked for during initialization), the function never advances past this point.

ethereum-optimism/optimism/op-interop-filter/filter/lockstep_cross_validator.go

Line 290 to 297 in be00aaa

```
290    currentTs := v.crossValidatedTs.Load()
291
292    // Lazy initialization: start from the configured start timestamp
293    if currentTs == 0 {
294        v.crossValidatedTs.Store(v.startTimestamp)
295        v.log.Info("Cross-validator initialized", "startTimestamp", v.startTimestamp)
296        return
297    }
```

Recommendation

Consider enforcing `startTimestamp != 0` in `NewLockstepCrossValidator` if it is not a valid value, or add a `bool` field to `LockstepCrossValidator` to explicitly mark if `crossValidatedTs` is initialized.

We also note that `NewLockstepCrossValidator` is called from `Service.initBackend`, where `startTimestamp` is initialized from the system clock, and is therefore not expected to be `0` in practice.

ethereum-optimism/optimism/op-interop-filter/filter/service.go

Line 153 to 158 in be00aaa

```
153  func (s *Service) initBackend(ctx context.Context, cfg *Config) error {
154    // Calculate start timestamp once for all components.
155    // Chain ingesters will start ingesting from (startTimestamp - backfillDuration)
156    // and report Ready() once they reach startTimestamp.
157    // Cross-validator initializes to startTimestamp and waits for chains to catch up.
158    startTimestamp := uint64(clock.SystemClock.Now().Unix())
```

ethereum-optimism/optimism/op-interop-filter/filter/service.go

Line 207 to 215 in be00aaa

```
207   crossValidator := NewLockstepCrossValidator(
208       ctx,
209       s.log,
210       s.metrics,
211       cfg.MessageExpiryWindow,
212       startTimestamp,
213       cfg.ValidationInterval,
214       chains,
215   )
```

Status

Fixed in https://github.com/ethereum-optimism/optimism/commit/03e803063f1770fefec785f22549442584ff1ed1.

Integer underflow in `calculateStartingBlock`

Severity: Low | Difficulty: Low | Recommended Action: Fix Code | Addressed by client

In `calculateStartingBlock`, `backfillTimestamp` underflows if `c.startTimestamp < uint64(backfillDuration.Seconds())`.

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 337 to 340 in be00aaa

```
337 // calculateStartingBlock returns the block number where ingestion should start,
338 // calculated from startTimestamp and backfillDuration.
339 func (c *LogsDBChainIngester) calculateStartingBlock() uint64 {
340   backfillTimestamp := c.startTimestamp - uint64(c.backfillDuration.Seconds())
```

Both fields are set directly from arguments of `NewLogsDBChainIngester`:

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 79 to 80 in be00aaa

```
79     startTimestamp uint64,
80     backfillDuration time.Duration,
```

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 123 to 124 in be00aaa

```
123       startTimestamp:   startTimestamp,
124       backfillDuration: backfillDuration,
```

In `service.go`, `startTimestamp` is set from the current system time, whereas `backfillDuration` comes from the `Config`.

ethereum-optimism/optimism/op-interop-filter/filter/service.go

Line 157 to 162 in 45c7305

```
157 func (s *Service) initBackend(ctx context.Context, cfg *Config) error {
158   // Calculate start timestamp once for all components.
159   // Chain ingesters will start ingesting from (startTimestamp - backfillDuration)
160   // and report Ready() once they reach startTimestamp.
161   // Cross-validator initializes to startTimestamp and waits for chains to catch up.
162   startTimestamp := uint64(clock.SystemClock.Now().Unix())
```

ethereum-optimism/optimism/op-interop-filter/filter/service.go

Line 199 to 200 in 45c7305

```
199             startTimestamp,
200             cfg.BackfillDuration,
```

Recommendation

Enforce `startTimestamp >= uint64(backfillDuration.Seconds())` in either `NewLogsDBChainIngester` or `calculateStartingBlock`, or log the underflow and start ingestion from the genesis block as a fallback.

Status

Fixed in https://github.com/ethereum-optimism/optimism/commit/e7b34fe3fb664cc4b1bbdbfa4602b99fb1fa843e.

Unchecked edge cases in `findAndSetEarliestBlock`

Severity: Medium | Difficulty: Low | Recommended Action: Fix Code | Not addressed by client

The function searches for the earliest block that can be opened in the `LogsDB` to populate `earliestIngestedBlock` with the earliest block whose logs can be queried (i.e. the block following the database root).

There are two edge cases however where it is not ensured that `LogsDB.OpenBlock` succeeds on the block that gets stored in `earliestIngestedBlock`, or that it even exists.

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 313 to 335 in be00aaa

```
313 func (c *LogsDBChainIngester) findAndSetEarliestBlock(latestBlock uint64) {
314   c.mu.RLock()
315   defer c.mu.RUnlock()
316
317   // Walk backward from latest to find the first block that can be opened.
318   // The anchor block (sealed but not fully ingested) will fail OpenBlock
319   // because it has no predecessor checkpoint data, so we'll identify
320   // the first block with actual log data.
321   earliest := latestBlock
322   for blockNum := latestBlock; blockNum > 0; blockNum-- {
323       _, _, _, err := c.logsDB.OpenBlock(blockNum)
324       if err != nil {
325           // This block can't be opened, the one after it is earliest queryable
326           earliest = blockNum + 1
327           break
328       }
329       earliest = blockNum
330   }
331
332   c.earliestIngestedBlock.Store(earliest)
333   c.earliestIngestedBlockSet.Store(true)
334   c.log.Info("Found earliest block in DB", "block", earliest)
335 }
```

Analysis

In each of the following cases, the function stores a block number in `earliestIngestedBlock` without checking that `OpenBlock` succeeds for the block.

Case 1: `OpenBlock(latestBlock)` fails

In this case, the iteration is terminated after the first iteration, and `latestBlock + 1` gets stored as the result. However, `latestBlock + 1` might not exist, but even if it does, the success of `OpenBlock(latestBlock + 1)` is not ensured.

Case 2: `latestBlock == 0`

In this case, the loop is not entered, and `OpenBlock` is not called. The value `0` is stored in `earliestIngestedBlock`, but `OpenBlock(0)` is never checked.

Call site

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 495 to 510 in be00aaa

```
495    // Check if we have existing data to resume from
496    c.mu.RLock()
497    latestSealed, hasSealed := c.logsDB.LatestSealedBlock()
498    c.mu.RUnlock()
499
500    if hasSealed {
501        // Resume from existing DB
502        nextBlock := latestSealed.Number + 1
503        c.log.Info("Resuming from existing DB", "lastSealed", latestSealed.Number, "resumeFrom",
nextBlock)
504
505        if !c.earliestIngestedBlockSet.Load() {
506            c.findAndSetEarliestBlock(latestSealed.Number)
507        }
508
509        return nextBlock, nil
510    }
```

Recommendation

For each case, establish whether its negation is a precondition of the function. If so, document it and if feasible enforce it using an explicit check. If not, fix the function for the given case.

Ingestion continues after setting error state

Severity: Low | Difficulty: Low | Recommended Action: Fix Code | Not addressed by client

When `ingestBlock` encounters an error, ingestion silently continues rather than halting, potentially processing blocks on top of a corrupt state.

Analysis

In `ingestBlock`, in all cases when the error state is set, the value `nil` is returned:

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 615 to 616 in be00aaa

```
615             c.SetError(ErrorReorg, fmt.Sprintf("parent hash mismatch at block %d", blockNum))
616             return nil
```

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 622 to 633 in be00aaa

```
622         if errors.Is(err, types.ErrConflict) {
623             c.SetError(ErrorConflict, fmt.Sprintf("database conflict at block %d", blockNum))
624             return nil
625         }
626         if errors.Is(err, types.ErrDataCorruption) {
627             c.SetError(ErrorDataCorruption, fmt.Sprintf("data corruption at block %d: %v", blockNum,
err))
628             return nil
629         }
630         if errors.Is(err, ErrInvalidLog) {
631             c.SetError(ErrorInvalidExecutingMessage, fmt.Sprintf("invalid log at block %d: %v",
blockNum, err))
632             return nil
633         }
```

On the call site, this causes the the inner loop to continue with the next block number (until `nextBlock <= head.NumberU64()`):

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 432 to 449 in be00aaa

```
432         // Inner loop: ingest all available blocks without waiting between them
433         for nextBlock <= head.NumberU64() {
434             // Check for shutdown between blocks
435             select {
436             case <-c.ctx.Done():
437                 return
438             default:
439             }
440
441             if err := c.ingestBlock(nextBlock); err != nil {
442                 // Application context was canceled (e.g., during shutdown).
443                 if errors.Is(err, context.Canceled) {
444                     return
445                 }
446                 c.log.Error("Failed to ingest block", "block", nextBlock, "err", err)
447                 break // Exit inner loop on error, wait for next tick to retry
448             }
449             nextBlock++
```

Within the loop `ingestBlock` is called again, returning immediately without an error:

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 583 to 585 in be00aaa

```
583     if c.Error() != nil {
584         return nil
585     }
```

The loop finally exits when `nextBlock > head.NumberU64()`, where the outer loop detects the error state.

Since after the first error `nextBlock` is incremented without actually ingesting blocks, `nextBlock == c.logsDB.LatestSealedBlock() + 1` is not an invariant of either the inner or the outer loop.

Recommendation

Return an error for the cases above. Then the inner loop breaks immediately, and the outer loop halts further ingestion by checking the error state.

Make `c.Error() != nil` an assertion in `ingestBlock` instead of a check.

In addition, for better locality, consider pulling the calls to `setError` from `checkReorg` and `ingestBlock` into `runIngestion`.

Informative Findings

This section includes observations that are not directly exploitable, but highlight areas for improvement in code clarity, maintainability, or best practices. While not critical, addressing these can strengthen the system's overall robustness.

Unofficial

`LockstepCrossValidator` informational findings

Severity: Informative | Not addressed by client

Consider enforcing `inclusionTimestamp <= execTimestamp` in `validateMessageTiming` if `timeout > 0`

As a result, all the events become ordered for the `timeout > 0` case:

```
   initTimestamp
<  inclusionTimestamp
<= execTimestamp
<  execTimestamp + timeout             // maxExecTimestamp
<= initTimestamp + messageExpiryWindow  // expiresAt
```

Alternatively, note that there's a single call site where `timeout > 0`, and there `execTimestamp == inclusionTimestamp`:

runtimeverification/_audits_Ethereum-optimism_optimism_interopv2/op-interop-filter/filter/lockstep_cross_validator.go

Line 196 to 205 in be00aaa

```
196   // Validate timing constraints (including timeout if set)
197   if err := validateMessageTiming(
198       access.Timestamp,
199       execDescriptor.Timestamp,
200       v.messageExpiryWindow,
201       execDescriptor.Timeout,
202       execDescriptor.Timestamp,
203   ); err != nil {
204       return err
205   }
```

By eliminating `execTimestamp`, the checks can be simplified, resulting in the following ordering for all `timeout` values:

```
   initTimestamp
<  inclusionTimestamp
<= inclusionTimestamp + timeout         // maxExecTimestamp
<= initTimestamp + messageExpiryWindow  // expiresAt
```

```
diff --git a/op-interop-filter/filter/lockstep_cross_validator.go b/op-interop-filter/filter/lockstep_cross_validator.go
```

```
index 9a9539c105..415d11f992 100644
--- a/op-interop-filter/filter/lockstep_cross_validator.go
+++ b/op-interop-filter/filter/lockstep_cross_validator.go
@@ -127,7 +127,7 @@ func (v *LockstepCrossValidator) CrossValidatedTimestamp() (uint64, bool) {
 //   - timeout: optional max execution delay (0 = disabled)
 //   - execTimestamp: execution timestamp (only used if timeout > 0)
 func validateMessageTiming(
-       initTimestamp, inclusionTimestamp, messageExpiryWindow, timeout, execTimestamp uint64,
+       initTimestamp, inclusionTimestamp, messageExpiryWindow, timeout uint64,
 ) error {
        // Rule 1: init must be strictly before inclusion
        if initTimestamp >= inclusionTimestamp {
@@ -142,26 +142,18 @@ func validateMessageTiming(
                        initTimestamp, messageExpiryWindow, types.ErrConflict)
        }

-       // Rule 3: message must not be expired at inclusion
-       if expiresAt < inclusionTimestamp {
-               return fmt.Errorf("initiating message expired: init %d + expiry window %d = %d <
inclusion %d: %w",
-                       initTimestamp, messageExpiryWindow, expiresAt, inclusionTimestamp,
types.ErrConflict)
+       // Rule 3: message must not expire before timeout deadline
+       maxExecTimestamp := inclusionTimestamp + timeout
+       if maxExecTimestamp < inclusionTimestamp {
+               return fmt.Errorf("overflow in max exec timestamp calculation: timestamp %d + timeout %d:
%w",
+                       inclusionTimestamp, timeout, types.ErrConflict)
        }
-
-       // Rule 4: if timeout set, message must not expire before timeout deadline
-       if timeout > 0 {
-               maxExecTimestamp := execTimestamp + timeout
-               if maxExecTimestamp < execTimestamp {
-                       return fmt.Errorf("overflow in max exec timestamp calculation: timestamp %d +
timeout %d: %w",
-                               execTimestamp, timeout, types.ErrConflict)
-               }
-               if expiresAt < maxExecTimestamp {
-                       return fmt.Errorf("initiating message will expire before timeout: "+
-                               "init %d + expiry %d = %d < exec %d + timeout %d = %d: %w",
-                               initTimestamp, messageExpiryWindow, expiresAt,
-                               execTimestamp, timeout, maxExecTimestamp,
-                               types.ErrConflict)
-               }
```

```
+       if expiresAt < maxExecTimestamp {
+               return fmt.Errorf("initiating message will expire before timeout: "+
+                       "init %d + expiry %d = %d < exec %d + timeout %d = %d: %w",
+                       initTimestamp, messageExpiryWindow, expiresAt,
+                       inclusionTimestamp, timeout, maxExecTimestamp,
+                       types.ErrConflict)
        }

        return nil
```

Consider checking for ingester errors in `getMinIngestedTimestamp`

Then errors will be checked in `ValidateAccessEntry` as well and not only in `advanceValidation`.

Also note that the `Ready()` check can be safely removed from `advanceValidation`, as it's also performed in `getMinIngesterTimestamp`.

```
diff --git a/op-interop-filter/filter/lockstep_cross_validator.go b/op-interop-
filter/filter/lockstep_cross_validator.go
index 9a9539c105..fb0b3d4c7d 100644
--- a/op-interop-filter/filter/lockstep_cross_validator.go
+++ b/op-interop-filter/filter/lockstep_cross_validator.go
@@ -272,16 +272,6 @@ func (v *LockstepCrossValidator) advanceValidation() {
                return
        }

-       // All chains must be ready and error-free
-       for _, ingester := range v.chains {
-               if ingester.Error() != nil {
-                       return
-               }
-               if !ingester.Ready() {
-                       return
-               }
-       }
-
        minIngestedTs, ok := v.getMinIngestedTimestamp()
        if !ok {
                return
@@ -349,6 +339,10 @@ func (v *LockstepCrossValidator) getMinIngestedTimestamp() (uint64, bool) {
        var minTs uint64
        first := true
        for _, ingester := range v.chains {
```

```
+               // All chains must be ready and error-free
+               if ingester.Error() != nil {
+                       return 0, false
+               }
                if !ingester.Ready() {
                        return 0, false
                }
```

`startTimestamp` is assumed cross-valid at initialization

At initialization, `crossValidatedTs` is set from `startTimestamp` without checking its cross-validity.

ethereum-optimism/optimism/op-interop-filter/filter/lockstep_cross_validator.go

Line 292 to 297 in be00aaa

```
292    // Lazy initialization: start from the configured start timestamp
293    if currentTs == 0 {
294        v.crossValidatedTs.Store(v.startTimestamp)
295        v.log.Info("Cross-validator initialized", "startTimestamp", v.startTimestamp)
296        return
297    }
```

Therefore, in this case, `crossValidatedTs` is not actually cross-validated.

Check cross-validity for `startTimestamp`, or document that the range of validated timestamps is `(startTimestamp, crossValidatedTs]`, i.e. the lower bound is exclusive.

`LogsDBChainIngester` informational findings

Severity: Informative | Not addressed by client

Consider reordering check and function call in `processBlockLogs`

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 660 to 663 in be00aaa

```
660    parentBlock := eth.BlockID{Hash: blockInfo.ParentHash(), Number: blockNum - 1}
661    if blockNum == 0 {
662        parentBlock = eth.BlockID{}
663    }
```

```
diff --git a/op-interop-filter/filter/logsdb_chain_ingester.go b/op-interop-
filter/filter/logsdb_chain_ingester.go
index 7db2d9a00a..43d2639556 100644
--- a/op-interop-filter/filter/logsdb_chain_ingester.go
+++ b/op-interop-filter/filter/logsdb_chain_ingester.go
@@ -657,9 +657,9 @@ func (c *LogsDBChainIngester) processBlockLogs(blockInfo eth.BlockInfo, blockID

    var logIndex uint32

-   parentBlock := eth.BlockID{Hash: blockInfo.ParentHash(), Number: blockNum - 1}
-   if blockNum == 0 {
-       parentBlock = eth.BlockID{}
+   parentBlock := eth.BlockID{}
+   if blockNum != 0 {
+       parentBlock = eth.BlockID{Hash: blockInfo.ParentHash(), Number: blockNum - 1}
    }

    for _, receipt := range receipts {
```

Missing `nil` check for `rollupCfg` in `NewLogsDBChainIngester`

In `NewLogsDBChainIngester`, `rollupCfg` is stored without a `nil` check:

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 126 in be00aaa

```
126         rollupCfg:        rollupCfg,
```

When calling `Start`, this results in a `nil` pointer dereference in `calculateStartingBlock`:

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 342 in be00aaa

```
342     startingBlock, err := c.rollupCfg.TargetBlockNumber(backfillTimestamp)
```

`ethClient` and `rpcClient` are not closed if `logsDB.Close()` returns an error

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 158 to 169 in be00aaa

```
158     if c.logsDB != nil {
159         if err := c.logsDB.Close(); err != nil {
160             return fmt.Errorf("failed to close logs DB: %w", err)
161         }
162     }
163
164     if c.ethClient != nil {
165         c.ethClient.Close()
166     }
167     if c.rpcClient != nil {
168         c.rpcClient.Close()
169     }
```

`SetError` discards `IsUint64` result on conversion

Therefore, if `chainID` does not fit into an `uint64`, the error to be set is recorded under the wrong chain ID.

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 192 to 195 in be00aaa

```
192    chainIDUint64, _ := c.chainID.Uint64()
193    if reason == ErrorReorg || reason == ErrorConflict {
194        c.metrics.RecordReorgDetected(chainIDUint64)
195    }
```

Temporary directory is not cleaned up if `LogsDB` creation fails

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 358 to 370 in be00aaa

```
358    } else {
359        tempDir, err := os.MkdirTemp("", fmt.Sprintf("interop-filter-chain-%s-*", c.chainID))
360        if err != nil {
361            return fmt.Errorf("failed to create temp directory: %w", err)
362        }
363        dbPath = filepath.Join(tempDir, "logs.db")
364        c.log.Warn("Using temporary directory for logs DB", "path", dbPath)
365    }
366
367    db, err := logs.NewFromFile(c.log, &logsDBMetrics{m: c.metrics, chainID: c.chainID}, c.chainID,
dbPath, true)
368    if err != nil {
369        return fmt.Errorf("failed to open logs DB: %w", err)
370    }
```

Consider swapping condition order

For readability and a minor performance benefit.

```
diff --git a/op-interop-filter/filter/logsdb_chain_ingester.go b/op-interop-
filter/filter/logsdb_chain_ingester.go
index 7db2d9a00a..a82c662b61 100644
--- a/op-interop-filter/filter/logsdb_chain_ingester.go
+++ b/op-interop-filter/filter/logsdb_chain_ingester.go
@@ -279,7 +279,7 @@ func (c *LogsDBChainIngester) GetExecMsgsAtTimestamp(timestamp uint64) ([]Includ
        }

        latestBlock, ok := c.logsDB.LatestSealedBlock()
-       if !c.earliestIngestedBlockSet.Load() || !ok {
```

```
+       if !ok || !c.earliestIngestedBlockSet.Load() {
                return nil, nil
        }
        earliest := c.earliestIngestedBlock.Load()
```

Consider using a write lock in `findAndSetEarliestBlock`

The function starts by acquiring a read lock.

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 313 to 315 in be00aaa

```
313  func (c *LogsDBChainIngester) findAndSetEarliestBlock(latestBlock uint64) {
314    c.mu.RLock()
315    defer c.mu.RUnlock()
```

However, it sets two fields, which in theory might interleave.

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 332 to 333 in be00aaa

```
332    c.earliestIngestedBlock.Store(earliest)
333    c.earliestIngestedBlockSet.Store(true)
```

(Note that `earliestIngestedBlockSet.Store(true)` is idempotent, therefore this is not a bug.)

Consider refactoring `sealParentBlock`

The function seals the block whose number it's passed to. Its naming on the other hand suggests that it's the parent of the block that gets sealed.

To make the function's purpose clear, consider renaming it's argument and local variables:

```
diff --git a/op-interop-filter/filter/logsdb_chain_ingester.go b/op-interop-
filter/filter/logsdb_chain_ingester.go
index 7db2d9a00a..c2522381d9 100644
--- a/op-interop-filter/filter/logsdb_chain_ingester.go
+++ b/op-interop-filter/filter/logsdb_chain_ingester.go
@@ -553,21 +553,21 @@ func (c *LogsDBChainIngester) checkReorg(head eth.BlockInfo) error {
    return fmt.Errorf("reorg detected")
 }

-func (c *LogsDBChainIngester) sealParentBlock(blockNum uint64) error {
-   c.log.Info("Sealing parent block as starting point", "block", blockNum)
```

```
+func (c *LogsDBChainIngester) sealParentBlock(parentNum uint64) error {
+   c.log.Info("Sealing parent block as starting point", "block", parentNum)

-   blockInfo, err := c.ethClient.InfoByNumber(c.ctx, blockNum)
+   parentInfo, err := c.ethClient.InfoByNumber(c.ctx, parentNum)
    if err != nil {
        return fmt.Errorf("failed to get block info: %w", err)
    }

-   blockID := eth.BlockID{Hash: blockInfo.Hash(), Number: blockInfo.NumberU64()}
+   parentID := eth.BlockID{Hash: parentInfo.Hash(), Number: parentInfo.NumberU64()}

    c.mu.Lock()
    defer c.mu.Unlock()

-   parentHash := blockInfo.ParentHash()
-   if err := c.logsDB.SealBlock(parentHash, blockID, blockInfo.Time()); err != nil {
+   grandParentHash := parentInfo.ParentHash()
+   if err := c.logsDB.SealBlock(grandParentHash, parentID, parentInfo.Time()); err != nil {
        return fmt.Errorf("failed to seal block: %w", err)
    }

@@ -575,7 +575,7 @@ func (c *LogsDBChainIngester) sealParentBlock(blockNum uint64) error {
    // an anchor checkpoint. earliestIngestedBlock will be set in ingestBlock when
    // the first block with actual log data is ingested.

-   c.log.Info("Sealed parent block", "block", blockNum, "hash", blockID.Hash)
+   c.log.Info("Sealed parent block", "block", parentNum, "hash", parentID.Hash)
    return nil
 }
```

or pulling the parent-specific log messages to the call site and generalizing the function:

```
diff --git a/op-interop-filter/filter/logsdb_chain_ingester.go b/op-interop-
filter/filter/logsdb_chain_ingester.go
index 7db2d9a00a..8e8618894d 100644
--- a/op-interop-filter/filter/logsdb_chain_ingester.go
+++ b/op-interop-filter/filter/logsdb_chain_ingester.go
@@ -510,10 +510,18 @@ func (c *LogsDBChainIngester) initIngestion() (uint64, error) {
    }

    // Fresh start: seal parent block as anchor
-   if err := c.sealParentBlock(startingBlock - 1); err != nil {
+   parentNum := startingBlock - 1
+   c.log.Info("Sealing parent block as starting point", "block", parentNum)
```

```
+
+   if err := c.sealBlockByNum(parentNum); err != nil {
        return 0, fmt.Errorf("failed to seal parent block: %w", err)
    }

+   // Note: We don't set earliestIngestedBlock here because the parent block is just
+   // an anchor checkpoint. earliestIngestedBlock will be set in ingestBlock when
+   // the first block with actual log data is ingested.
+   c.log.Info("Sealed parent block", "block", parentNum)
+
    c.log.Info("Starting fresh ingestion",
        "from", startingBlock,
        "to", head.NumberU64(),
@@ -553,9 +561,7 @@ func (c *LogsDBChainIngester) checkReorg(head eth.BlockInfo) error {
    return fmt.Errorf("reorg detected")
 }

-func (c *LogsDBChainIngester) sealParentBlock(blockNum uint64) error {
-   c.log.Info("Sealing parent block as starting point", "block", blockNum)
-
+func (c *LogsDBChainIngester) sealBlockByNum(blockNum uint64) error {
    blockInfo, err := c.ethClient.InfoByNumber(c.ctx, blockNum)
    if err != nil {
        return fmt.Errorf("failed to get block info: %w", err)
@@ -571,11 +577,6 @@ func (c *LogsDBChainIngester) sealParentBlock(blockNum uint64) error {
        return fmt.Errorf("failed to seal block: %w", err)
    }

-   // Note: We don't set earliestIngestedBlock here because the parent block is just
-   // an anchor checkpoint. earliestIngestedBlock will be set in ingestBlock when
-   // the first block with actual log data is ingested.
-
-   c.log.Info("Sealed parent block", "block", blockNum, "hash", blockID.Hash)
    return nil
 }
```

Consider sharing the `LogsDB` interface between `op-supernode` and `op-interop-filter`

Both `op-supernode` and `op-interop-filter` are backed by `logs.DB` from `op-supervisor`, but they handle the dependency inconsistently:

- `op-supernode` (to be precise, the `interop` activity) defines and depends on a `LogsDB` interface

- `op-interop-filter` (`LogsDBChainIngester`) depends directly on the concrete implementation

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go
Line 49 in be00aaa

```
49      logsDB          *logs.DB
```

Since `op-interop-filter` (aside from `LogsDBChainIngester.initLogsDB`) only depends on methods covered by the interface, the interface could be moved to a shared location (e.g. `op-supervisor`) so both packages can depend on the abstraction rather than the implementation. This would improve testability and make the dependency boundary explicit.

Consider extracting a function for the DB resume logic in `initIngestion`

```
diff --git a/op-interop-filter/filter/logsdb_chain_ingester.go b/op-interop-
filter/filter/logsdb_chain_ingester.go
index 7db2d9a00a..7b5736d5ea 100644
--- a/op-interop-filter/filter/logsdb_chain_ingester.go
+++ b/op-interop-filter/filter/logsdb_chain_ingester.go
@@ -471,6 +471,11 @@ func (c *LogsDBChainIngester) runIngestion() {

 // initIngestion performs one-time setup and returns the first block to ingest.
 func (c *LogsDBChainIngester) initIngestion() (uint64, error) {
+   nextBlock, resumed := c.tryResumeFromExistingDB()
+   if resumed {
+       return nextBlock, nil
+   }
+
    head, err := c.ethClient.InfoByLabel(c.ctx, eth.Unsafe)
    if err != nil {
        return 0, fmt.Errorf("failed to get current head: %w", err)
@@ -492,23 +497,6 @@ func (c *LogsDBChainIngester) initIngestion() (uint64, error) {

    c.log.Info("Determined starting block", "block", startingBlock, "startTimestamp", c.startTimestamp)

-   // Check if we have existing data to resume from
-   c.mu.RLock()
-   latestSealed, hasSealed := c.logsDB.LatestSealedBlock()
-   c.mu.RUnlock()
-
-   if hasSealed {
-       // Resume from existing DB
```

```
-       nextBlock := latestSealed.Number + 1
-       c.log.Info("Resuming from existing DB", "lastSealed", latestSealed.Number, "resumeFrom",
nextBlock)
-
-       if !c.earliestIngestedBlockSet.Load() {
-           c.findAndSetEarliestBlock(latestSealed.Number)
-       }
-
-       return nextBlock, nil
-   }
-
    // Fresh start: seal parent block as anchor
    if err := c.sealParentBlock(startingBlock - 1); err != nil {
        return 0, fmt.Errorf("failed to seal parent block: %w", err)
@@ -522,6 +510,27 @@ func (c *LogsDBChainIngester) initIngestion() (uint64, error) {
    return startingBlock, nil
 }

+func (c *LogsDBChainIngester) tryResumeFromExistingDB() (uint64, bool) {
+   // Check if we have existing data to resume from
+   c.mu.RLock()
+   latestSealed, hasSealed := c.logsDB.LatestSealedBlock()
+   c.mu.RUnlock()
+
+   if !hasSealed {
+       return 0, false
+   }
+
+   // Resume from existing DB
+   nextBlock := latestSealed.Number + 1
+   c.log.Info("Resuming from existing DB", "lastSealed", latestSealed.Number, "resumeFrom", nextBlock)
+
+   if !c.earliestIngestedBlockSet.Load() {
+       c.findAndSetEarliestBlock(latestSealed.Number)
+   }
+
+   return nextBlock, true
+}
+
 // checkReorg checks if a reorg occurred when head moves behind our progress.
 func (c *LogsDBChainIngester) checkReorg(head eth.BlockInfo) error {
    headNum := head.NumberU64()
```

Inconsistent computation of starting block in `initIngestion`

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 480 to 491 in be00aaa

```
480    startingBlock := c.calculateStartingBlock()
481
482    // Clamp to head if needed
483    if startingBlock > head.NumberU64() {
484        startingBlock = head.NumberU64()
485    }
486
487    // Guard against underflow: genesis block has no parent to seal
488    if startingBlock == 0 {
489        startingBlock = 1
490        c.log.Info("Starting from block 1 (genesis has no parent to seal)")
491    }
```

If `head.NumberU64() == 0` or `c.calculateStartingBlock() == 0`, the result of the function is `1`, i.e. `min(head.NumberU64(), c.calculateStartingBlock()) + 1`. Otherwise, it's `min(head.NumberU64(), c.calculateStartingBlock())`.

Consider making the two cases consistent:

```
diff --git a/op-interop-filter/filter/logsdb_chain_ingester.go b/op-interop-
filter/filter/logsdb_chain_ingester.go
index 7db2d9a00a..5b793e8db5 100644
--- a/op-interop-filter/filter/logsdb_chain_ingester.go
+++ b/op-interop-filter/filter/logsdb_chain_ingester.go
@@ -477,19 +477,7 @@ func (c *LogsDBChainIngester) initIngestion() (uint64, error) {
    }
    c.log.Info("Current chain head", "block", head.NumberU64(), "hash", head.Hash())

-   startingBlock := c.calculateStartingBlock()
-
-   // Clamp to head if needed
-   if startingBlock > head.NumberU64() {
-       startingBlock = head.NumberU64()
-   }
-
-   // Guard against underflow: genesis block has no parent to seal
-   if startingBlock == 0 {
-       startingBlock = 1
-       c.log.Info("Starting from block 1 (genesis has no parent to seal)")
-   }
+   startingBlock := min(head.NumberU64(), c.calculateStartingBlock()) + 1

    c.log.Info("Starting ingestion", "startingBlock", startingBlock)
```

```
-   }
-
+   startingBlock := min(head.NumberU64(), c.calculateStartingBlock()) + 1
    c.log.Info("Determined starting block", "block", startingBlock, "startTimestamp", c.startTimestamp)

    // Check if we have existing data to resume from
```

This also makes it obvious that there is a previous block for `sealParentBlock` to seal in the database. The function can even be refactored to avoid the subtraction `startingBlock - 1` altogether, and instead do

```
rootBlock := min(head.NumberU64(), c.calculateStartingBlock())
// ...
if err := c.sealParentBlock(rootBlock); err != nil {
   // ...
}
// ...
return rootBlock + 1, nil
```

`hasLatest` should be asserted instead of branched on in `ingestBlock`

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 599 to 604 in be00aaa

```
599    c.mu.RLock()
600    latestBlock, hasLatest := c.logsDB.LatestSealedBlock()
601    c.mu.RUnlock()
602
603    // Always verify parent hash when we have a previous block
604    if hasLatest {
```

Reason:

1. After a successful call to `initIngestion`, there should be at least one sealed block in the `LogsDB`.
2. `ingestBlock` is only called if `initIngestion` succeeds.
3. The `LogsDB` is only appended to, in particular, `Rewind` is never called.

Consider implementing `findAndSetEarliestBlock` in terms of `LogsDB.FirstSealedBlock`

The current implementation is based on an iteration and multiple database queries.

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 313 to 335 in be00aaa

```go
func (c *LogsDBChainIngester) findAndSetEarliestBlock(latestBlock uint64) {
  c.mu.RLock()
  defer c.mu.RUnlock()

  // Walk backward from latest to find the first block that can be opened.
  // The anchor block (sealed but not fully ingested) will fail OpenBlock
  // because it has no predecessor checkpoint data, so we'll identify
  // the first block with actual log data.
  earliest := latestBlock
  for blockNum := latestBlock; blockNum > 0; blockNum-- {
      _, _, _, err := c.logsDB.OpenBlock(blockNum)
      if err != nil {
          // This block can't be opened, the one after it is earliest queryable
          earliest = blockNum + 1
          break
      }
      earliest = blockNum
  }

  c.earliestIngestedBlock.Store(earliest)
  c.earliestIngestedBlockSet.Store(true)
  c.log.Info("Found earliest block in DB", "block", earliest)
}
```

But `logs.DB` might already implement the same or a similar functionality:

ethereum-optimism/optimism/op-supernode/supernode/activity/interop/logdb.go

Line 25 to 26 in be00aaa

```go
    // FirstSealedBlock returns the first block seal in the DB.
    FirstSealedBlock() (types.BlockSeal, error)
```

Avoid subtraction in `runIngestion`

This is not a bug, since `nextBlock` should be at least `1`:

- It initialized from `initIngestion`, which should return at least `1`.
- It is only ever incremented in the function.

Nevertheless, it is easy to make the absence of underflow immediately obvious:

```
diff --git a/op-interop-filter/filter/logsdb_chain_ingester.go b/op-interop-
filter/filter/logsdb_chain_ingester.go
index 7db2d9a00a..f003840b24 100644
--- a/op-interop-filter/filter/logsdb_chain_ingester.go
+++ b/op-interop-filter/filter/logsdb_chain_ingester.go
@@ -423,7 +423,7 @@ func (c *LogsDBChainIngester) runIngestion() {
        }

        // Reorg detection: if head moved behind our progress, check hash
-       if head.NumberU64() < nextBlock-1 {
+       if head.NumberU64()+1 < nextBlock {
            if err := c.checkReorg(head); err != nil {
                continue
            }
```

`checkReorg` logs spurious error if `earliestIngestedBlock` is unset

If `!c.earliestIngestedBlockSet.Load()`, `checkReorg` performs `c.BlockHashAt(headNum)`.

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 527 to 541 in be00aaa

```go
527    headNum := head.NumberU64()
528
529    // If head is before our earliest block, we can't verify - this is expected
530    if c.earliestIngestedBlockSet.Load() && headNum < c.earliestIngestedBlock.Load() {
531        c.log.Debug("Head before our earliest block, can't verify",
532            "head", headNum, "earliest", c.earliestIngestedBlock.Load())
533        return nil
534    }
535
536    dbHash, ok := c.BlockHashAt(headNum)
537    if !ok {
538        // We should have this block but can't get it - unexpected
539        c.log.Error("Failed to get block hash for reorg verification", "block", headNum)
540        return nil
541    }
```

Since on the call site `headNum < nextBlock - 1`, where `nextBlock - 1` is the root of the `LogsDB` in this case, `c.BlockHashAt(headNum)` fails and logs an error.

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 425 to 430 in be00aaa

```
425         // Reorg detection: if head moved behind our progress, check hash
426         if head.NumberU64() < nextBlock-1 {
427             if err := c.checkReorg(head); err != nil {
428                 continue
429             }
430         }
```

Instead, `c.earliestIngestedBlockSet.Load()` should be checked separately.

```
diff --git a/op-interop-filter/filter/logsdb_chain_ingester.go b/op-interop-
filter/filter/logsdb_chain_ingester.go
index 7db2d9a00a..15a47e7574 100644
--- a/op-interop-filter/filter/logsdb_chain_ingester.go
+++ b/op-interop-filter/filter/logsdb_chain_ingester.go
@@ -526,8 +526,13 @@ func (c *LogsDBChainIngester) initIngestion() (uint64, error) {
 func (c *LogsDBChainIngester) checkReorg(head eth.BlockInfo) error {
    headNum := head.NumberU64()

+   if !c.earliestIngestedBlockSet.Load() {
+       c.log.Debug("Earliest block has not been set yet, can't verify", "head", headNum)
+       return nil
+   }
+
    // If head is before our earliest block, we can't verify - this is expected
-   if c.earliestIngestedBlockSet.Load() && headNum < c.earliestIngestedBlock.Load() {
+   if headNum < c.earliestIngestedBlock.Load() {
        c.log.Debug("Head before our earliest block, can't verify",
            "head", headNum, "earliest", c.earliestIngestedBlock.Load())
        return nil
```

After the fix, `ealiestIngestedBlock.Load() <= headNum < nextBlock - 1` and thus `headNum` should be present in the database. Therefore, `c.BlockHashAt(headNum)` should pass, so the failure case should be modified to set the error state as it indicates a DB inconsistency that has to be resolved manually.

No output if the error is set

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 410 to 413 in be00aaa

```
410         // Skip if in error state
411         if c.Error() != nil {
412             continue
413         }
```

Consider periodically logging a warning message to indicate that the ingester is still operational.

Consider adding a `Rewind()` method

Currently, if the ingester enters an error state, `LogsDB` consistency must be restored manually and the ingester restarted.

Consider adding a `Rewind()` method that rolls the DB back to a known-good state and optionally clears the error, enabling recovery without a full restart. This also moves the ingester closer to a model where it owns the DB, preventing inconsistent external changes.

Document in `processBlockLogs` that `execMsg == nil` is a valid case

In `processBlockLogs`, the result of `processors.DecodeExecutingMessageLog` is checked for errors.

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 669 to 672 in be00aaa

```
669             execMsg, err := processors.DecodeExecutingMessageLog(l)
670             if err != nil {
671                 return 0, fmt.Errorf("invalid log %d in block %d: %w: %w", l.Index, blockNum,
ErrInvalidLog, err)
672             }
```

However, there are cases when `execMsg == nil`, even though `err == nil`.

ethereum-optimism/optimism/op-supervisor/supervisor/backend/processors/executing_message.go

Line 31 to 35 in 53d3861

```
31  func DecodeExecutingMessageLog(l *ethTypes.Log) (*types.ExecutingMessage, error) {
32      msg, err := MessageFromLog(l)
33      if err != nil || msg == nil {
34          return nil, err
35      }
```

ethereum-optimism/optimism/op-supervisor/supervisor/backend/processors/executing_message.go

Line 14 to 23 in 53d3861

```go
func MessageFromLog(l *ethTypes.Log) (*types.Message, error) {
 if l.Address != params.InteropCrossL2InboxAddress {
     return nil, nil
 }
 if len(l.Topics) != 2 { // topics: event-id and payload-hash
     return nil, nil
 }
 if l.Topics[0] != types.ExecutingMessageEventTopic {
     return nil, nil
 }
```

In such a case, `AddLog` is called with `nil` as `execMsg`.

ethereum-optimism/optimism/op-interop-filter/filter/logsdb_chain_ingester.go

Line 674 to 676 in be00aaa

```go
        if err := c.logsDB.AddLog(logHash, parentBlock, logIndex, execMsg); err != nil {
            return 0, fmt.Errorf("failed to add log: %w", err)
        }
```

Add a comment to indicate that this is a valid case.

Supernode supports same-timestamp messages, but Interop-Filter rejects them

Severity: Informative | Not addressed by client

Currently, the Interop-Filter rejects executing messages that refer to initiating messages at the same timestamp:

runtimeverification/_audits_Ethereum-optimism_optimism_interopv2/op-interop-filter/filter/lockstep_cross_validator.go

Line 133 to 136 in be00aaa

```
133    if initTimestamp >= inclusionTimestamp {
134        return fmt.Errorf("initiating message timestamp %d not before inclusion timestamp %d: %w",
135            initTimestamp, inclusionTimestamp, types.ErrConflict)
136    }
```

This means that, if we expect Sequencers to run the Interop-Filter as part of the block creation process, it will not allow any blocks to be created with same-timestamp interop messages. However, the Supernode does expect the possibility of same-timestamp messages, only rejecting executing messages that refer to initiating messages from a future timestamp (and additionally implementing a check to ensure that same-timestamp messages don't form a cycle in their dependency graph):

runtimeverification/_audits_Ethereum-optimism_optimism_interopv2/op-supernode/supernode/activity/interop/algo.go

Line 161 to 164 in be00aaa

```
161    if execMsg.Timestamp > executingTimestamp {
162        return fmt.Errorf("initiating timestamp %d > executing timestamp %d: %w",
163            execMsg.Timestamp, executingTimestamp, ErrTimestampViolation)
164    }
```

This is intended behavior for the time being, with interop being rolled out gradually. For now, the Interop-Filter is not expected to support same-timestamp messages, but in the future the interop mechanism will be updated to allow interop messages at the granularity of Flashblocks, so that an executing message can refer to an initiating message in the same block but a previous Flashblock. The Supernode is already implemented to support this future update.

Supernode code quality suggestions

Severity: Informative | Not addressed by client

`interop.go`

- `commitVerifiedResult()` doesn't use its `timestamp` parameter. Additionally, since all it does is calling `i.verifiedDB.Commit(verifiedResult)`, the function could just be removed in favor of calling `Commit()` directly.
- `decideVerifiedResult()` doesn't use its `RoundObservation` parameter.
- `progressInterop()` returns a `RoundObservation` along with its `StepOutput`, but this return value seems to only be used in the caller to obtain the `LastVerifiedTS`. Since this can just be obtained directly from the `VerifiedDB`, the `RoundObservation` return value could be removed, which would simplify some of the function signatures.
- `refreshCurrentL1OnWait()` returns `(false, nil)` on every path, making the return value redundant. Instead, it could just not return anything and have `progressAndRecord()` return `(false, nil)` after calling it.
- `observeRound()` returns an incomplete `RoundObservation` when it returns with an error. This is not presently a problem since the only caller discards the return value in this case, but it would probably be preferable to return an empty value in this case to prevent partial values from being used accidentally.
- The `LastVerified` field of `RoundObservation` is not used anywhere outside of `observeRound()`, so it could be a local variable instead of part of the struct.
- In `verify()`, instead of passing `view` as an argument to `verifyFn()` and `cycleVerifyFn()` directly, it is stored in the `frontierView` field of the `Interop` struct, which is then read by the functions. It would be preferable to explicitly pass it as a parameter to the functions instead.
- `cycleVerifyFn()` returns a full `Result`, but the only field that is used from it is `InvalidHeads`. Instead of building a full `Result`, the function could return only the invalid heads.
- `applyPendingTransition()` sorts `invalidations` by `ChainID` before calling `invalidateBlock`, but it's unclear why this would be necessary. The same applies to `applyRewindPlan()` when sorting the `ChainID` list.
- `applyRewindPlan()` defines a local function `recordErr()`, but it seems unnecessary compared to simply calling `append()` directly. Additionally, `recordErr()` checks if `err != nil`, but this is already checked previously everywhere it is called.
- `checkChainsReady()` goes over all the results and records the first error, then returns this error if it's not `nil`. Instead, it could simply return immediately as soon as it finds an error.

verified_db.go

- In `Commit()`, the nested if statements for `v.initialized` and `ts != v.lastTimestamp+1` can be combined into a single `if v.initialized && ts != v.lastTimestamp+1`.
- `Commit()` and `Get()` use two different procedures for copying `val` to a variable outside of the DB transaction (`Commit()` uses `append`, while `Get()` uses `copy`). It would be better to pick one approach and use it consistently.
- In `GetPendingTransition()`, `val` is copied into `data`, which is then unmarshalled. Instead, `val` could be unmarshalled directly.
- `initLastTimestamp()` silently returns no error if the bucket for the verified results doesn't exist. However, it should always be the case that the bucket already exists when `initLastTimestamp()` is called. If it doesn't, something went wrong and it would be better for the function to return an error.

algo.go

- The `blockPerChain` type alias slightly obscures the original type `map[eth.ChainID]eth.BlockID` without adding much information. It might be more clear to just use the original type.
- `verifyExecutingMessage()` ignores the `executingChain` and `logIdx` parameters.
- `verifyInteropMessages()` first checks the `frontierView` for the latest block from a given chain, then falls back to the `LogsDB` if the block is not present in the `frontierView`. However, by construction, the `frontierView` should contain a block for every chain, so the fallback is unnecessary. If a block is missing, this should be considered an error.
- The `blocksAtTimestamp` parameter in `verifyInteropMessages()` is unnecessary, since the relevant information is already present in `frontierView`. This also makes the check `blockRef.Hash != expectedBlock.Hash` unnecessary (the `frontierBlock` matches the `expectedBlock` hash by construction, and the case where we fall back to the `LogsDB` can be removed, as described above).

rewind.go

- In `computeRewindTargets()`, there is no reason to call `earliest()` for the finalized head, since we already return an error if the target block is greater than the current finalized.
- `RewindToTimestamp()` converts the given timestamp to a block number to perform the rewind. However, in `InvalidateBlock()` this timestamp is computed by converting it from a block number in the first place. If `RewindToTimestamp()` were replaced by a function that receives the block number instead, this back-and-forth conversion could be avoided.

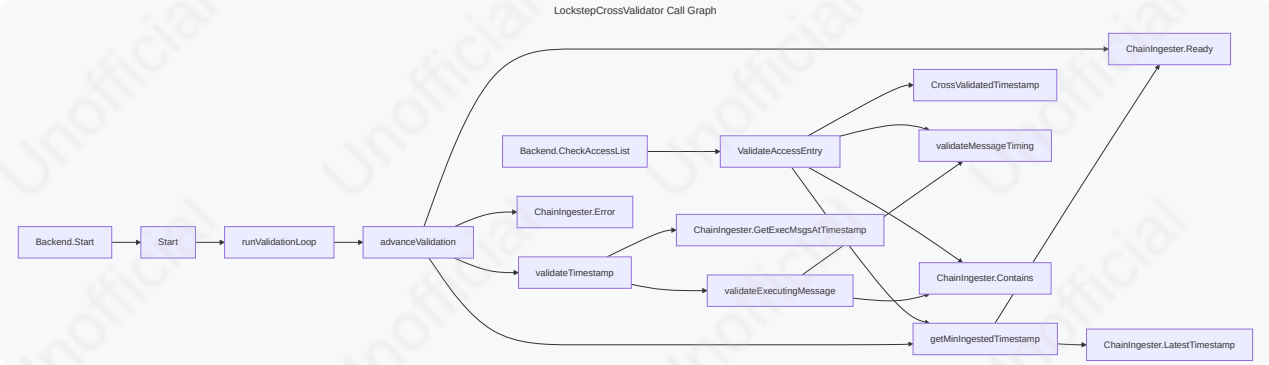
Appendix

The following appendix encompasses more detailed technical reports and artifacts produced during the course of the audit.

Unofficial

`LockstepCrossValidator` Function Summaries

Call Graph



Function Summaries

The following section is a simplified summary of function postconditions for `LockstepCrossValidator`.
We apply the following simplifying assumptions:

- We assume each function to evaluate in an instant, in particular, we do not consider race conditions arising from asynchronous `ChainIngester` behavior.
- Since we're not reasoning about concurrency, we treat `crossValidatedTs` as `uint64`.

`validateMessageTiming`

```
func validateMessageTiming(
    initTimestamp       uint64,
    inclusionTimestamp  uint64,
    messageExpiryWindow uint64,
    timeout             uint64,
    execTimestamp       uint64,
) (err error)
```

Summary

- If `timeout == 0`, then `err == nil` iff
  - `initTimestamp < inclusionTimestamp <= initTimestamp + messageExpiryWindow`
- If `timeout > 0`, then `err == nil` iff
  - `initTimestamp < inclusionTimestamp <= initTimestamp + messageExpiryWindow`

- `execTimestamp < execTimestamp + timeout <= initTimestamp + messageExpiryWindow`

`getMinIngestedTimestamp`

```
func (v *LockstepCrossValidator) getMinIngestedTimestamp() (ts uint64, ok bool)
```

Summary

- `ok` iff
  - `len(v.chains) > 0`
  - `forall (ingester : v.chains)`
    - `ingester.Ready()`
    - `ingester.LatestTimestamp().ok`
- If `ok` then `ts == min (ingester : v.chains) ingester.LatestTimestamp().ts`
- If `!ok` then `ts == 0`

`CrossValidatedTimestamp`

```
func (v *LockstepCrossValidator) CrossValidatedTimestamp() (ts uint64, init bool)
```

Summary

- `ts == v.crossValidatedTs`
- `init` iff `ts != 0`

`ValidateAccessEntry`

```
func (v *LockstepCrossValidator) ValidateAccessEntry(
    access types.Access,
    minSafety types.SafetyLevel,
    execDescriptor types.ExecutingDescriptor,
) (err error)
```

Summary

- `err == nil` iff
  - `v.getMinIngestedTimestamp().ok`
  - `v.getMinIngestedTimestamp().ts >= access.Timestamp`
  - if `minSafety == types.CrossUnsafe` then `v.CrossValidatedTimestamp().init`

- if `minSafety == types.CrossUnsafe` then
  `v.CrossValidatedTimestamp().ts >= access.Timestamp`
- ```
  validateMessageTiming(
      access.Timestamp,          // initTimestamp
       execDescriptor.Timestamp,  // inclusionTimestamp
       v.messageExpiryWindow,     // messageExpiryWindow
       execDescriptor.Timeout,    // timeout
       execDescriptor.Timestamp,  // execTimestamp
  ).err == nil
  ```
- `v.chains[access.ChainID].ok`
- ```
  ingester.Contains(types.ContainsQuery{
      Timestamp: access.Timestamp,
      BlockNum:  access.BlockNumber,
      LogIdx:    access.LogIndex,
      Checksum:  access.Checksum,
  }).err == nil
  ```

  where `ingester == v.chains[access.ChainID].value`

After substitutions:

- `err == nil` iff
  - `len(v.chains) > 0`
  - `forall (ingester : v.chains)`
    - `ingester.Ready()`
    - `ingester.LatestTimestamp().ok`
  - `min (ingester : v.chains) ingester.LatestTimestamp().ts >= access.Timestamp`
  - if `minSafety == types.CrossUnsafe` then
    - `v.crossValidatedTs != 0`
    - `v.crossValidatedTs >= access.Timestamp`
  - ```
    access.Timestamp
    <  execDescriptor.Timestamp
    <= execDescriptor.Timestamp + execDescriptor.Timeout
    <= access.Timestamp + v.messageExpiryWindow
    ```
  - `v.chains[access.ChainID].ok`
  - ```
    i.Contains(types.ContainsQuery{
        Timestamp: access.Timestamp,
    ```

```
    BlockNum:  access.BlockNumber,
    LogIdx:    access.LogIndex,
    Checksum:  access.Checksum,
}).err == nil
```

where `i == v.chains[access.ChainID].value`

`validateExecutingMessage`

```
func (v *LockstepCrossValidator) validateExecutingMessage(
    execMsg *types.ExecutingMessage,
    inclusionTimestamp uint64,
) (err error)
```

Summary

- `err == nil` iff
  - `v.chains[execMsg.ChainID].ok`
  - ```
    validateMessageTiming(
        execMsg.Timestamp,      // initTimestamp
        inclusionTimestamp,     // inclusionTimestamp
        v.messageExpiryWindow,  // messageExpiryWindow
        0,                      // timeout
        0,                      // execTimestamp
    ).err == nil
    ```
  - ```
    ingester.Contains(types.ContainsQuery{
        Timestamp: execMsg.Timestamp,
        BlockNum:  execMsg.BlockNum,
        LogIdx:    execMsg.LogIdx,
        Checksum:  execMsg.Checksum,
    }).err == nil
    ```

    where `ingester == v.chains[execMsg.ChainID].value`

After substitutions:

- `err == nil` iff
  - `v.chains[execMsg.ChainID].ok`
  - `execMsg.Timestamp < inclusionTimestamp <= execMsg.Timestamp + v.messageExpiryWindow`
  - ```
    ingester.Contains(types.ContainsQuery{
        Timestamp: execMsg.Timestamp,
    ```
    ```

```
    BlockNum:  execMsg.BlockNum,
    LogIdx:    execMsg.LogIdx,
    Checksum:  execMsg.Checksum,
}).err == nil
```

where `ingester == v.chains[execMsg.ChainID].value`

`validateTimestamp`

```
func (v *LockstepCrossValidator) validateTimestamp(ts uint64) (err error)
```

Summary

- `err == nil` iff
  `forall (ingester : v.chains)`
  - `ingester.GetExecMsgsAtTimestamp(ts).err == nil`
  - `forall (msg : ingester.GetExecMsgsAtTimestamp(ts).msgs) v.validateExecutingMessage(msg.ExecutingMessage, msg.InclusionTimestamp).err == nil`

After substitutions:

- `err == nil` iff
  `forall (ingester : v.chains)`
  - `ingester.GetExecMsgsAtTimestamp(ts).err == nil`
  - `forall (msg : ingester.GetExecMsgsAtTimestamp(ts).msgs)`
    - `v.chains[msg.ExecutingMessage.ChainID].ok`
    - `msg.ExecutingMessage.Timestamp < msg.InclusionTimestamp <= msg.ExecutingMessage.Timestamp + v.messageExpiryWindow`
    - ```
      i.Contains(types.ContainsQuery{
          Timestamp: msg.ExecutingMessage.Timestamp,
          BlockNum:  msg.ExecutingMessage.BlockNum,
          LogIdx:    msg.ExecutingMessage.LogIdx,
          Checksum:  msg.ExecutingMessage.Checksum,
      }).err == nil
      ```

where `i == v.chains[msg.ExecutingMessage.ChainID].value`
      ```

`advanceValidation`

```
func (v *LockstepCrossValidator) advanceValidation()
```

Summary

- If either
  - `v.Error() != nil`
  - `exists (ingester : v.chains)` where either
    - `ingester.Error() != nil`
    - `!ingester.Ready()`
  - `!v.getMinIngestedTimestamp().ok`
  - `v.crossValidatedTs == 0` and `v.startTimestamp == 0` (see `LockstepCrossValidator` never advances if `startTimestamp` is `0`)
  - `v.crossValidatedTs >= v.getMinIngestedTimestamp().ts`

  then

  ```
  next(v) == v
  ```

- Otherwise, if `v.crossValidatedTs == 0` and `v.startTimestamp != 0`, then

  ```
  next(v) == { ...v, crossValidatedTs: v.startTimestamp }
  ```

- Otherwise, if

  ```
  forall (ts : [v.crossValidatedTs + 1 .. v.getMinIngestedTimestamp().ts]) v.validateTimestamp(ts)
  == nil
  ```

  then

  ```
  next(v) == { ...v, crossValidatedTs: v.getMinIngestedTimestamp().ts }
  ```

- Otherwise, let minimal

  ```
  ts : [v.crossValidatedTs + 1 .. v.getMinIngestedTimestamp().ts]
  ```

  such that `v.validateTimestamp(ts) != nil`. Then

  ```
  next(v) == { ...v, crossValidatedTs: ts - 1, err: e }
  ```

  for some `e != nil`

After substitutions:

- If either
  - `v.Error() != nil`
  - `len(v.chains) == 0`
  - `exists (ingester : v.chains)` where either
    - `ingester.Error() != nil`
    - `!ingester.Ready()`
    - `!ingester.LatestTimestamp().ok`
  - `v.crossValidatedTs == 0` and `v.startTimestamp == 0`
  - `v.crossValidatedTs >= min (ingester : v.chains) ingester.LatestTimestamp().ts`

  then

  `next(v) == v`
- Otherwise, if `v.crossValidatedTs == 0` and `v.startTimestamp != 0`, then

  `next(v) == { ...v, crossValidatedTs: v.startTimestamp }`
- Otherwise, if

  `forall (ts : [v.crossValidatedTs + 1 .. min (ingester : v.chains) ingester.LatestTimestamp().ts]) (ingester : v.chains)`
  - `ingester.GetExecMsgsAtTimestamp(ts).err == nil`
  - `forall (msg : ingester.GetExecMsgsAtTimestamp(ts).msgs)`
    - `v.chains[msg.ExecutingMessage.ChainID].ok`
    - `msg.ExecutingMessage.Timestamp < msg.InclusionTimestamp <= msg.ExecutingMessage.Timestamp + v.messageExpiryWindow`
    - ```
      i.Contains(types.ContainsQuery{
          Timestamp: msg.ExecutingMessage.Timestamp,
          BlockNum:  msg.ExecutingMessage.BlockNum,
          LogIdx:    msg.ExecutingMessage.LogIdx,
          Checksum:  msg.ExecutingMessage.Checksum,
      }).err == nil
      ```

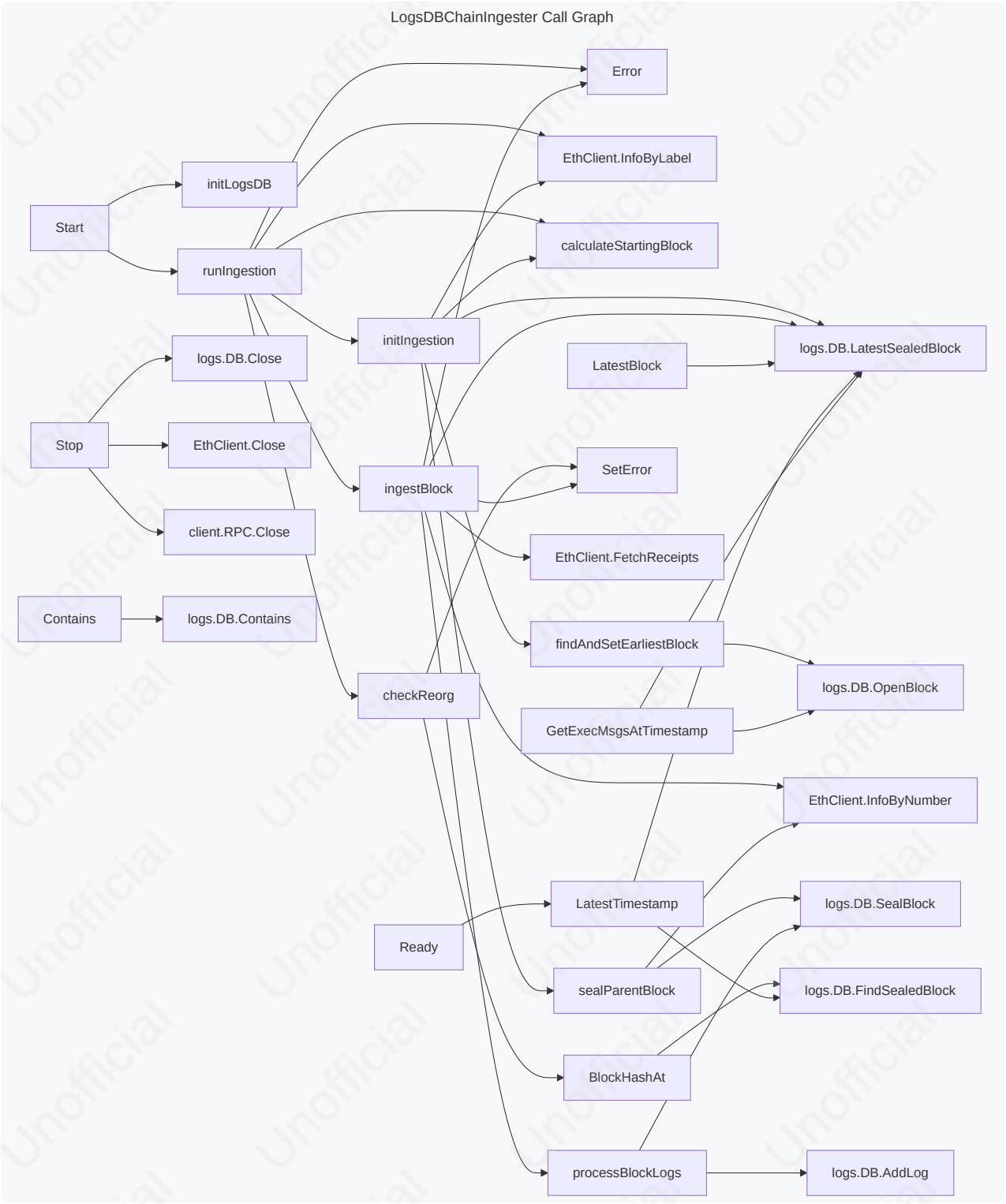
      where `i == v.chains[msg.ExecutingMessage.ChainID].value`

  then
      ```

`next(v) == { ...v, crossValidatedTs: min (ingester : v.chains) ingester.LatestTimestamp().ts }`

- Otherwise, let minimal

  `ts : [v.crossValidatedTs + 1 .. min (ingester : v.chains) ingester.LatestTimestamp().ts]`

  such that

  `exists (ingester : v.chains)`

  where either

  - `ingester.GetExecMsgsAtTimestamp(ts).err != nil`
  - `exists (msg : ingester.GetExecMsgsAtTimestamp(ts).msgs)`

    where either

    - `!v.chains[msg.ExecutingMessage.ChainID].ok`
    - `msg.ExecutingMessage.Timestamp >= msg.InclusionTimestamp`
    - `msg.InclusionTimestamp > msg.ExecutingMessage.Timestamp + v.messageExpiryWindow`
    - ```
      i.Contains(types.ContainsQuery{
          Timestamp: msg.ExecutingMessage.Timestamp,
          BlockNum:  msg.ExecutingMessage.BlockNum,
          LogIdx:    msg.ExecutingMessage.LogIdx,
          Checksum:  msg.ExecutingMessage.Checksum,
      }).err != nil
      ```

      where `i == v.chains[msg.ExecutingMessage.ChainID].value`

  Then

  `next(v) == { ...v, crossValidatedTs: ts - 1, err: e }`

  for some `e != nil`.
      ```

`LogsDBChainIngester` Function Summaries

Call Graph

Unofficial


LogsDBChainIngester Call Graph
Error
EthClient.InfoByLabel
initLogsDB
Start
calculateStartingBlock
runIngestion
initIngestion
logs.DB.LatestSealedBlock
logs.DB.Close
LatestBlock
Stop
EthClient.Close
SetError
ingestBlock
client.RPC.Close
EthClient.FetchReceipts
Contains
logs.DB.Contains
findAndSetEarliestBlock
logs.DB.OpenBlock
checkReorg
GetExecMsgsAtTimestamp
EthClient.InfoByNumber
LatestTimestamp
logs.DB.SealBlock
Ready
sealParentBlock
logs.DB.FindSealedBlock
BlockHashAt
processBlockLogs
logs.DB.AddLog

Function Summaries

`NewLogsDBChainIngester`

```
func NewLogsDBChainIngester(
    parentCtx context.Context,
    logger log.Logger,
    m metrics.Metricer,
    chainID eth.ChainID,
    rpcURL string,
    dataDir string,
    startTimestamp uint64,
    backfillDuration time.Duration,
    pollInterval time.Duration,
    rollupCfg *rollup.Config,
) (*LogsDBChainIngester, error)
```

Creates and initializes a `LogsDBChainIngester`, setting up an `RPC` client and the `EthClient` wrapping it. Notably, the `LogsDB` is left uninitialized until a subsequent call to `Start`.

Error handling

- Calls `cancel` if the RPC client cannot be instantiated
- Calls `rpcClient.Close` and `cancel` if the `EthClient` cannot be initialized

Errors eventually propagate to the top and get reported by the CLI:

ethereum-optimism/optimism/op-interop-filter/cmd/main.go

Line 46 to 48 in be00aaa

```
46      if err != nil {
47          log.Crit("Application failed", "message", err)
48      }
```

Callers

`Service.initBackend`

ethereum-optimism/optimism/op-interop-filter/filter/service.go

Line 188 to 202 in be00aaa

```
188         ingester, err := NewLogsDBChainIngester(
189             ctx,
190             s.log,
191             s.metrics,
192             chainID,
193             rpcURL,
194             cfg.DataDir,
195             startTimestamp,
196             cfg.BackfillDuration,
197             cfg.PollInterval,
198             rollupCfg,
199         )
200         if err != nil {
201             return fmt.Errorf("failed to create chain ingester for chain %s: %w", chainID, err)
202         }
```

Called by `Service.init`

ethereum-optimism/optimism/op-interop-filter/filter/service.go

Line 99 to 101 in be00aaa

```
99    if err := s.initBackend(ctx, cfg); err != nil {
100       return fmt.Errorf("failed to init backend: %w", err)
101   }
```

Called by `NewService` :

ethereum-optimism/optimism/op-interop-filter/filter/service.go

Line 84 to 86 in be00aaa

```
84    if err := s.init(ctx, cfg); err != nil {
85        return nil, errors.Join(err, s.Stop(ctx))
86    }
```

Called by `Main` :

ethereum-optimism/optimism/op-interop-filter/cmd/main.go

Line 36 in be00aaa

```
36      app.Action = cliapp.LifecycleCmd(filter.Main(app.Version))
```

ethereum-optimism/optimism/op-service/cliapp/lifecycle.go

Line 46 to 53 in be00aaa

```
46          appLifecycle, err := fn(ctx, appCancel)
47          if err != nil {
48              // join errors to include context cause (nil errors are dropped)
49              return errors.Join(
50                  fmt.Errorf("failed to setup: %w", err),
51                  context.Cause(ctx.Context),
52              )
53          }
```

ethereum-optimism/optimism/op-interop-filter/filter/service.go

Line 74 in be00aaa

```
74          return NewService(cliCtx.Context, cfg, l)
```

`Start`

```
func (c *LogsDBChainIngester) Start() error
```

Initializes the `LogsDB` and launches the background ingestion goroutine.

Error handling

- Returns an error if the `LogsDB` cannot be initialized.

Errors eventually propagate to the top and get reported by the CLI:

ethereum-optimism/optimism/op-interop-filter/cmd/main.go

Line 46 to 48 in be00aaa

```
46      if err != nil {
47          log.Crit("Application failed", "message", err)
48      }
```

Callers

Called by `Backend.Start` :

ethereum-optimism/optimism/op-interop-filter/filter/backend.go

Line 60 to 64 in be00aaa

```
60    for chainID, ingester := range b.chains {
61        if err := ingester.Start(); err != nil {
62            return fmt.Errorf("failed to start chain ingester for %v: %w", chainID, err)
63        }
64    }
```

Called by `Service.Start` :

ethereum-optimism/optimism/op-interop-filter/filter/service.go

Line 286 to 289 in be00aaa

```
286    // Start backend (begins block ingestion)
287    if err := s.backend.Start(ctx); err != nil {
288        return fmt.Errorf("failed to start backend: %w", err)
289    }
```

Called by `LifecycleCmd` :

ethereum-optimism/optimism/op-interop-filter/cmd/main.go

Line 36 in be00aaa

```
36      app.Action = cliapp.LifecycleCmd(filter.Main(app.Version))
```

ethereum-optimism/optimism/op-service/cliapp/lifecycle.go

Line 55 to 61 in be00aaa

```
55        if err := appLifecycle.Start(appCtx); err != nil {
56            // join errors to include context cause (nil errors are dropped)
57            return errors.Join(
58                fmt.Errorf("failed to start: %w", err),
59                context.Cause(appCtx),
60            )
61        }
```

Stop

```
func (c *LogsDBChainIngester) Stop() error
```

Gracefully stops the ingester: marks as stopped (for idempotency), calls `cancel` , waits for the ingestion goroutine to finish, then closes the `LogsDB` , `EthClient` , and `RPC` under write lock.

Error handling

- Returns an error if the `LogsDB` cannot be closed.

Errors eventually propagate to the top and get reported by the CLI:

ethereum-optimism/optimism/op-interop-filter/cmd/main.go

Line 46 to 48 in be00aaa

```
46      if err != nil {
47          log.Crit("Application failed", "message", err)
48      }
```

Callers

Called by `Backend.Stop`:

ethereum-optimism/optimism/op-interop-filter/filter/backend.go

Line 84 to 88 in be00aaa

```
84      for chainID, ingester := range b.chains {
85          if err := ingester.Stop(); err != nil {
86              result = errors.Join(result, fmt.Errorf("failed to stop chain ingester for %v: %w",
chainID, err))
87          }
88      }
```

Called by `Service.Start` on error:

ethereum-optimism/optimism/op-interop-filter/filter/service.go

Line 291 to 296 in be00aaa

```
291     // Start main RPC server (supervisor API)
292     if err := s.rpcServer.Start(); err != nil {
293         // Rollback: stop backend if RPC server fails to start
294         stopErr := s.backend.Stop(ctx)
295         return errors.Join(fmt.Errorf("failed to start RPC server: %w", err), stopErr)
296     }
```

ethereum-optimism/optimism/op-interop-filter/filter/service.go

Line 301 to 306 in be00aaa

```
301         if err := s.adminRPCServer.Start(); err != nil {
302             // Rollback: stop main RPC and backend if admin RPC server fails
303             rpcErr := s.rpcServer.Stop()
304             backendErr := s.backend.Stop(ctx)
305             return errors.Join(fmt.Errorf("failed to start admin RPC server: %w", err), rpcErr,
backendErr)
306         }
```

Or by `Service.Stop`:

ethereum-optimism/optimism/op-interop-filter/filter/service.go

Line 332 to 336 in be00aaa

```
332     if s.backend != nil {
333         if err := s.backend.Stop(ctx); err != nil {
334             result = errors.Join(result, fmt.Errorf("failed to stop backend: %w", err))
335         }
336     }
```

Which is called by `LifecycleCmd`:

ethereum-optimism/optimism/op-interop-filter/cmd/main.go

Line 36 in be00aaa

```
36      app.Action = cliapp.LifecycleCmd(filter.Main(app.Version))
```

ethereum-optimism/optimism/op-service/cliapp/lifecycle.go

Line 70 to 80 in be00aaa

```
70          // Execute graceful stop.
71          stopErr := appLifecycle.Stop(stopCtx)
72          // note: Stop implementation may choose to suppress a context error,
73          // if it handles it well (e.g. stop idling after a halt).
74          if stopErr != nil {
75              // join errors to include context cause (nil errors are dropped)
76              return errors.Join(
77                  fmt.Errorf("failed to stop: %w", stopErr),
78                  context.Cause(stopCtx),
79              )
80          }
```

`Ready`

```
func (c *LogsDBChainIngester) Ready() bool
```

Returns `true` if the latest sealed block's timestamp (`LatestTimestamp`) is at least the `startTimestamp` .

Error handling

- Returns `false` if `LatestTimestamp()` fails

`SetError`

Atomically stores an `IngesterError` , logs it, and if the reason is `ErrorReorg` or `ErrorConflict` then records a reorg using the `Metricer` .

```
func (c *LogsDBChainIngester) SetError(reason IngesterErrorReason, msg string)
```

Callers

`checkReorg` , `ingestBlock` , both of which are called from `runIngestion` .

`Error`

```
func (c *LogsDBChainIngester) Error() *IngesterError
```

Returns the current error state.

`ClearError`

```
func (c *LogsDBChainIngester) ClearError()
```

Atomically clears the error state.

Callers

None.

`Contains`

```
func (c *LogsDBChainIngester) Contains(query types.ContainsQuery) (types.BlockSeal, error)
```

Checks containment in the `LogsDB` .

Callers

`validateExecutingMessage` and `ValidateAccessEntry` in `LockstepCrossValidator`.

Error handling

- Returns `ErrUninitialized` if `logsDB` is `nil`. (Never the case if `Start` completes without error.)
- The function calls `LogsDB.Contains`, which calls `panic` in certain cases:

ethereum-optimism/optimism/op-supervisor/supervisor/backend/db/logs/db.go

Line 334 to 336 in be00aaa

```
334    if err == nil {
335        panic("expected iterator to stop with error")
336    }
```

`LatestBlock`

```
func (c *LogsDBChainIngester) LatestBlock() (eth.BlockID, bool)
```

Returns `(id, true)` with `id` the ID of the latest sealed block in the `LogsDB` if the DB is initialized such a block exists, and `(BlockID{}, false)` otherwise.

Callers

None.

Error handling

- Returns `(BlockID{}, false)` if `logsDB` is `nil`.

`BlockHashAt`

```
func (c *LogsDBChainIngester) BlockHashAt(blockNum uint64) (common.Hash, bool)
```

Returns `(hash, true)` with the `hash` for a sealed block with the given number by looking it up in the `LogsDB` if the DB is initialized and such a block exists.

Callers

Called by `checkReorg`. If the call fails, `checkReorg` logs an error.

Error handling

- Returns `(Hash{}, false)` if the DB is uninitialized or the query fails.
- The function calls `LogsDB.FindSealedBlock`, which calls `panic` in certain cases:

ethereum-optimism/optimism/op-supervisor/supervisor/backend/db/logs/db.go

Line 162 to 172 in be00aaa

```
162    h, n, ok := iter.SealedBlock()
163    if !ok {
164        panic("expected block")
165    }
166    if n != number {
167        panic(fmt.Sprintf("found block seal %s %d does not match expected block number %d", h, n,
number))
168    }
169    timestamp, ok := iter.SealedTimestamp()
170    if !ok {
171        panic("expected timestamp")
172    }
```

`LatestTimestamp`

```
func (c *LogsDBChainIngester) LatestTimestamp() (uint64, bool)
```

Returns the timestamp of the latest sealed block by performing two consecutive `LogsDB` queries:

1. `LogsDB.LatestSealedBlock` returns the `BlockID` of the latest sealed block
2. `LogsDB.FindSealedBlock`, looks up the sealed block by number, from which the timestamp is extracted

Callers

Called by `Ready`, which returns `false` if the call fails, propagating the failure.

Error handling

- Returns `(0, false)` if `logsDB` is `nil`, or if any of the queries fail.
- The function calls `LogsDB.FindSealedBlock`, which calls `panic` in certain cases.

`GetExecMsgsAtTimestamp`

```
func (c *LogsDBChainIngester) GetExecMsgsAtTimestamp(timestamp uint64) ([]IncludedMessage, error)
```

Scans all blocks between the earliest ingested block and the latest sealed block in the `LogsDB` (terminating early once `timestamp` is surpassed) to collect all executing messages from inclusion blocks with `timestamp`.

Callers

`LockstepCrossValidator.validateTimestamp` . A log entry is emitted if the call fails with an error.

Error handling

- Returns `ErrUninitialized` if `logsDB` is `nil` .
- Calls `LogsDB.OpenBlock` , which calls `LogsDB.newIteratorAt` , which calls `panic` in certain cases:

ethereum-optimism/optimism/op-supervisor/supervisor/backend/db/logs/db.go

Line 437 to 440 in be00aaa

```
437         h, num, ok := iter.SealedBlock()
438         if !ok {
439             panic("expected sealed block")
440         }
```

`findAndSetEarliestBlock`

```
func (c *LogsDBChainIngester) findAndSetEarliestBlock(latestBlock uint64)
```

Walks backward from `latestBlock` to find the earliest block that can be successfully opened via `LogsDB.OpenBlock` , then atomically stores the result into `earliestIngestedBlock` and sets the `earliestIngestedBlockSet` flag.

Related issue: Unchecked edge cases in `findAndSetEarliestBlock` .

Callers

`initIngestion` if ingestion is resumed for an existing `LogsDB` . Called with the number of the latest sealed block as `latestBlock` .

Error handling

- Calls `LogsDB.OpenBlock` , which `panic` s in certain cases.

`calculateStartingBlock`

```
func (c *LogsDBChainIngester) calculateStartingBlock() uint64
```

Calculates the block number from which ingestion should start, by subtracting `backfillDuration` from `startTimestamp` to get `backfillTimestamp` , then using `rollupCfg` to resolve that timestamp to a block number.

Related issue: Integer undeflow in `calculateStartingBlock`.

Callers

`initIngestion` and `runIngestion`.

Error handling

- If `backfillTimestamp` is before genesis, the result is the number of the genesis block.

`initLogsDB`

```
func (c *LogsDBChainIngester) initLogsDB() error
```

Initializes the logs database. If `dataDir` is set, creates a chain-specific subdirectory and opens the DB there. Otherwise creates a temporary directory and warns. Stores the opened DB into `c.logsDB` under the write lock.

Callers

`Start`, which propagates any potential error in the call.

Error handling

- Returns an error if the `LogsDB` cannot be created.

`runIngestion`

```
func (c *LogsDBChainIngester) runIngestion()
```

The main ingestion function.

1. Initializes the `LogsDB` by calling `initIngestion`, which sets `nextBlock` to the first block to ingest (`nextBlock - 1` is the latest sealed block in the DB).
2. Outer loop: runs with a period time `pollInterval`.
   2.1. If the error state is set, continue the loop.
   2.2. Fetch the unsafe `head` with `EthClient.InfoByLabel`.
   2.3. If `head.NumberU64() < nextBlock - 1`, call `checkReorg` to check for a reorg.
   2.4. Inner loop: ingest up until `head` (`for nextBlock <= head.NumberU64() { ... }`). Call `ingestBlock(nextBlock)`, then increment `nextBlock`.

Related issue: Ingestion continues after setting error state.

Callers

`Start`, as a gorutine.

Error handling

- An error in `initIngestion` causes the function to return.
- If the error state is set, the outer loop continues without any effect, halting further ingestion until the error is cleared.
- If `EthClient.InfoByLabel` fails, in the general case, the outer loop is continued, resulting in a retry in the next iteration. If the error is `context.Canceled`, the function returns.
- If `checkReorg` returns with an error, the outer loop is continued. Since in that case the error state is also set, ingestion halts.
- If `ingestBlock` returns with an error, in the general case, the inner loop is broken out of, resulting in a retry. If the error is `context.Canceled`, the function returns. The function also potentially sets the error state.

`initIngestion`

```
func (c *LogsDBChainIngester) initIngestion() (uint64, error)
```

Ensures the `LogsDB` has a root block (without any logs added). Returns the number of the first block to ingest. Has two disjoint code paths.

1. `LogsDB` already has a sealed block (i.e. `LatestSealedBlock` succeeds)
   - The result is the block number following the latest sealed block.
   - As a side effect, if `!earliestIngestedBlockSet.Load()`, `findAndSetEarliestBlock` is called on the number of the latest sealed block to initialize `earliestIngestedBlock` (and `earliestIngestedBlockSet`).

2. Otherwise, the `LogsDB` needs to be initialized with a root.
   - The unsafe head, `head`, is queried using `EthClient.InfoByLabel`.
   - `startingBlock`, the first block to ingest, is `max(1, min(head.NumberU64(), c.calculateStartingBlock()))`.
     - It cannot be earlier than block `1` to ensure it has a parent. (`head` is at least the genesis block, otherwise, `EthClient.InfoByLabel` should have failed.)
     - It cannot be later than `head`, except if `head` is the genesis block, in which case it is block `1`.
   - `c.sealParentBlock(startingBlock - 1)`: create a root for the chain in `LogsDB`.
   - The result is `startingBlock`.

Callers

Called at the beginning of `runIngestion`. Returned errors are logged and cause `runIngestion` to return, i.e., stop the ingestion gorutine.

Error handling

- Returns an error if `EthClient.InfoByLabel` or `sealParentBlock` fails.

`checkReorg`

```
func (c *LogsDBChainIngester) checkReorg(head eth.BlockInfo) error
```

Checks whether a reorg occurred by comparing the supplied chain `head`, against the block with the corresponding number in the `LogsDB`. Sets error state and returns error on mismatch.

A reorg is detected if the following conditions hold:

1. `earliestIngestedBlock` is not set, or is set to at most `head.NumberU64()`
2. `c.BlockHashAt(head.NumberU64())` sucessfully looks up `dbHash`, the block hash for the corresponding block number in the `LogsDB`
3. `dbHash != head.Hash()`

Callers

`runIngestion`, outer loop. In case of an error, the loop is continued. Since this only happens if the error state has been set, the next iteration blocks further ingestion.

Error handling

- Logs an error if the block hash for the head block's number cannot be fetched from the database.
- Sets and returns an error if a reorg is detected.

`sealParentBlock`

```
func (c *LogsDBChainIngester) sealParentBlock(blockNum uint64) error
```

Fetches the block info for a given block number using the `EthClient` and seals it in the `LogsDB`, using its actual parent hash.

Callers

`initIngestion`, which propagates any potential error.

Error handling

- Returns an error if `ethClient.InfoByNumber` fails
- Returns an error if `logsDB.SealBlock` fails

`ingestBlock`

```
func (c *LogsDBChainIngester) ingestBlock(blockNum uint64) error
```

Ingests a single block with number `blockNum` by fetching its info and receipts using the `EthClient`, verifies block number and parent hash relative to the latest block stored in the `LogsDB`, and calls `processBlockLogs` to add logs to the block and seal it in the `LogsDB`. Sets the earliest ingested block if there's none.

Callers

`runIngestion`, inner loop. If the call returns an error, the ingestion gorutine is terminated if is `context.Canceled`, otherwise the error is logged and the gorutine continues with the next tick.

Error handling

- If the error is set, the function is a no-op.
- Propagates errors from `EthClient.InfoByNumber` and `EthClient.FetchReceipts`.
- If `blockNum` does not equal the number of the last sealed block plus one, an error is returned.
- If the the parent hash of block number `blockNum` does not match the hash of the last sealed block, the error state is set, and `nil` is returned.
- If `processBlockLogs` fails with `ErrConflict`, `ErrDataCorruption`, or `ErrorInvalidExecutingMessage`, the error state is set, and `nil` is returned. Any other error from the call is returned.

`processBlockLogs`

```
func (c *LogsDBChainIngester) processBlockLogs(
    blockInfo eth.BlockInfo,
    blockID eth.BlockID,
    receipts gethTypes.Receipts,
    blockNum uint64
) (uint32, error)
```

Under a write lock, iterates over all the logs in `receipts`, decodes them as executing messages and adds them to the `LogsDB` via `LogsDB.AddLog`.

Finally, block `blockID` is sealed.
Returns the number of logs processed if successful.

Related issue: `processBlockLogs` potentially calls `AddLog` with `nil` as `execMsg`.

Callers

`ingestBlock`, which handles errors from the called function as follows:

- If the error is `ErrConflict`, `ErrDataCorruption` or `ErrInvalidLog`, call `SetError` to set the error state.
- Otherwise, return the error.
  - On the call site (`runIngestion`), if the error is `context.Canceled`, return from the ingestion loop.
  - Otherwise, log the error, break from the inner loop, and wait for the next iteration to retry.

Error handling

- Returns an error if `DecodeExecutingMessageLog`, `LogsDB.AddLog` or `LogsDB.SealBlock` fails.

Dafny Model of the Supernode

As part of the audit, we have created a model of the Supernode in Dafny, a programming language with formal verification capabilities. Dafny allows the programmer to specify `requires` and `ensures` clauses for every function, expressing that function's preconditions and postconditions respectively. It is then able to automatically verify statically at compile time that these function contracts are satisfied, using a powerful underlying reasoning engine based on SMT solvers.

Our Dafny model of the Supernode can be found at https://github.com/runtimeverification/_audits_Ethereum-optimism_optimism_interopv2/tree/rv/dafny-models/op-supernode/dafny-models. Note that this model is not fully equivalent to the Supernode implementation in Go, and instead aims to capture the overall logic of the Supernode's cross-validation loop and state transition. The goals of this model are the following:

- Serve as a reference implementation for the Supernode with which the Go implementation can be compared to identify possible discrepancies.
- Capture the logic of certain parts of the Supernode in a clearer way that is not obscured by implementation details.
- Suggest ways to refactor the code to improve code quality.
- Clarify our understanding of the code and its underlying logic, facilitating manual analysis.
- Help identify and specify preconditions, postconditions, invariants and other properties that the correctness of the Supernode depends on.
- Allow proving via Dafny's automated formal verification that these desirable properties and invariants hold in the model, therefore increasing our confidence that they hold in the implementation itself.

Components

- A full model of the `VerifiedDB` that provably maintains the following invariants:
  - The DB is either empty, with no entries and no last verified timestamp, or all entries have sequential timestamps with no gaps, and the last verified timestamp is equal to the highest timestamp present in the DB.
  - Every entry in the DB has an L2 head for every chain ID.
  - If there is a pending transition, that pending transition satisfies certain internal consistency checks (e.g. timestamps are greater than 0, chain IDs match the ones used by the DB).
- Abstracted models of the `LogsDB` and `ChainContainer`, where methods are left unimplemented but with their expected behavior specified as postconditions.
- A model of the Supernode where (with some caveats, see below) `progressAndRecord()` provably preserves the following invariants IF the state transition is successfully applied (that is,

`applyPendingTransition()` doesn't fail in the middle of a state update):
    - The invariants of the `VerifiedDB` hold.
    - The `LogsDBs` are sequential: every block is the successor of the previous block (block number increases by 1, parent hash matches the previous block's hash, and timestamp increases).
    - All timestamps in the `LogsDB` are less than or equal to the last verified timestamp in `VerifiedDB`.
- Individual functions have their own preconditions and postconditions that are also checked by Dafny.

Caveats

- If the state transition is not successfully applied in full, it is stored in the `VerifiedDB` as a pending transition and is re-tried in the next round until it succeeds. Until that happens, the Supernode might be in an intermediate state where only part of the state has been updated. The model does not yet prove that re-applying the pending transition successfully recovers to a fully consistent state.
- Some extra consistency checks have been added to the model to ensure that there is consistency between the blocks fetched from the L2 chains, the blocks in the `VerifiedDB` and the blocks in the `LogsDBs`. Some of these checks could be removed if we add stronger assumptions and invariants to the model (allowing Dafny's verifier to prove that they always hold under our assumptions), but it might also be worth adding these to the implementation as sanity checks.
- The invariant proofs depend on the specification of the `LogsDB` and `ChainContainer` being correct. Some of the assumptions we make might not hold in the current implementation; for example, that an update to the `LogsDB` leaves it unchanged in case of error (see #49). If these don't match, either it's a bug that needs to be fixed in the implementation or the model should be corrected to align with the actual implementation behavior.
- The same-timestamp cycle check implemented in `verifyCycleMessages()` is not currently modeled, and is left abstracted.